

Universidade Federal de Catalão
Instituto de Biotecnologia
Curso de Bacharelado em Ciência da Computação

Predição da cotação Real/Bitcoin usando a rede
neural Long Short Term Memory (LSTM)

Ramon Soares Mendes Meneses Leite

Ramon Soares Mendes Meneses Leite

Predição da cotação Real/Bitcoin usando a rede neural Long Short Term Memory (LSTM)

Monografia apresentada ao Curso de Bacharelado em Ciência da Computação da Universidade Federal de Catalão, como parte dos requisitos para obtenção do grau de Bacharel em Ciência da Computação. *VERSÃO REVISADA*

Orientador: Prof. Dr. Sérgio Francisco da Silva

Catalão – GO
2023

Ficha de identificação da obra elaborada pelo autor, através do
Programa de Geração Automática do Sistema de Bibliotecas da UFG.

Leite, Ramon Soares Mendes Meneses

Predição da cotação Real/Bitcoin usando a rede neural Long
Short Term Memory (LSTM) [manuscrito] / Ramon Soares Mendes
Meneses Leite. – 2023.

64 p.: il.

Orientador: Prof. Dr. Sérgio Francisco da Silva

Monografia (Graduação) – Universidade Federal de Catalão,
Instituto de Biotecnologia, Ciência da Computação, 2023.

Bibliografia.

1. Redes neurais recorrentes. 2. memória de curto a longo prazo.
3. cotação de criptomoedas. 4. predição. 5. regressão multivariável.
- I. Silva, Sérgio Francisco da, orient. II. Título.

CDU 004

Ramon Soares Mendes Meneses Leite

Predição da cotação Real/Bitcoin usando a rede neural Long Short Term Memory (LSTM)

Monografia apresentada ao curso de
Bacharelado em Ciência da Computação da
Universidade Federal de Catalão.

Trabalho aprovado em 03 de Março de 2023.

Sérgio Francisco da Silva
Orientador

Núbia Rosa da Silva
Universidade Federal de Catalão

Tércio Alberto dos Santos Filho
Universidade Federal Catalão

Catalão – GO
2023

RESUMO

LEITE, R. S. M. M.. **Predição da cotação Real/Bitcoin usando a rede neural Long Short Term Memory (LSTM)**. 2023. 64 p. Monografia (Graduação) – Instituto de Biotecnologia, Universidade Federal de Catalão – , Catalão – GO.

Uma criptomoeda é um tipo de dinheiro digital onde as transações são realizadas de forma eletrônica, as finanças são descentralizadas e todos os usuários podem usufruir dos serviços sem intervenção de terceiros. Como todo ativo financeiro, uma questão importante para investimentos é a sua previsibilidade. Assim, previsões temporais de cotação de criptomoedas têm-se tornado importante para estimular pessoas físicas, empresas e governos a investir neste tipo ativo, a minimizar perdas e, possivelmente, ter lucro. Adicionalmente, devido à gerência descentralizada, dados temporais de transação de criptomoedas são abertos, o que permite usar séries de dados históricas para estudos e geração de modelos de predição de cotações. Este trabalho analisa o desempenho de uma técnica de aprendizado de máquina proeminente, a rede neural *long short term memory* (LSTM), para a predição cotação Real/Bitcoin usando múltiplas variáveis temporais e a compara a rede neural recorrente simples. O problema é modelado como um problema de regressão, onde o objetivo é prever a taxa de conversão do Real Brasileiro para a criptomoeda Bitcoin em um dado horizonte. Também são investigados vários aspectos dos modelos, como predição para um único horizonte, predição para horizontes múltiplos, capacidade para realizar previsões de curto a longo prazo, além de estabelecer uma comparação dos modelos. Ao final dos experimentos foi possível constatar que o melhor modelo de previsão foi o LSTM de saída única para horizonte 1, onde o erro absoluto médio foi de 2,58%.

Palavras-chave: Redes neurais recorrentes, memória de curto a longo prazo, cotação de criptomoedas, predição, regressão multivariável.

ABSTRACT

LEITE, R. S. M. M.. **Predição da cotação Real/Bitcoin usando a rede neural Long Short Term Memory (LSTM)**. 2023. 64 p. Monografia (Graduação) – Instituto de Biotecnologia, Universidade Federal de Catalão – , Catalão – GO.

A cryptocurrency is a type of digital money where transactions are carried out electronically, finances are decentralized and all users can enjoy the services without the intervention of third parties. Like any financial asset, an important issue for investments is its predictability. Thus, temporal predictions of cryptocurrency prices have become important to encourage individuals, companies and governments to invest in this type of asset, to minimize losses and, possibly, make profit. Additionally, due to decentralized management, temporal cryptocurrency transaction data is open, which allows using historical data series for studies and generation of price prediction models. This study analyzes the performance of a prominent machine learning technique, the *long short term memory* (LSTM) neural network, for Real/Bitcoin price prediction using multiple temporal variables and compares it to the simple recurrent neural network. The problem is modeled as a regression problem, where the objective is to predict the conversion rate from the Brazilian Real to the Bitcoin cryptocurrency in a given horizon. Several aspects of the models are also investigated, such as prediction for a single horizon, prediction for multiple horizons, ability to make short-term and long-term predictions, in addition to establishing a comparison of the models. At the end of the experiments, it was possible to verify that the best forecasting model was the single output LSTM for horizon 1, where the mean absolute error was 2.58%.

Keywords: Recurrent neural networks, long short-term memory, cryptocurrencies quote, multi-variable regression.

LISTA DE ILUSTRAÇÕES

Figura 1 – Formação dos candles.	22
Figura 2 – Série temporal da produção anual dos EUA de queijos azul e gorgonzola. . .	23
Figura 3 – Série temporal dos embarques mensais de produtos da fabricante de bebidas dos EUA, não ajustados.	23
Figura 4 – Série temporal do preço de fechamento do Bitcoin.	26
Figura 5 – Decomposição série temporal.	27
Figura 6 – Tendência de alta no Bitcoin.	27
Figura 7 – Tendência de baixa no Bitcoin.	28
Figura 8 – Padrão diferente de uma tendência no Bitcoin.	28
Figura 9 – SVR com margem	32
Figura 10 – Exemplo de rede neural recorrente.	34
Figura 11 – Ilustração de um neurônio.	36
Figura 12 – Ilustração de um neurônio artificial.	37
Figura 13 – Arquitetura <i>feedforward</i>	38
Figura 14 – Cálculo do gradiente e <i>back-propagation</i>	39
Figura 15 – Estrutura de uma rede neural recorrente.	40
Figura 16 – RNR estendida	40
Figura 17 – RNR - Estrutura de cadeia e módulo de repetição	41
Figura 18 – LSTM - Estrutura de cadeia e módulo de repetição	41
Figura 19 – Modelo LSTM.	49
Figura 20 – Modelo LSTM - Preço real vs previsão	51
Figura 21 – Modelo RNN - Preço real vs previsão	53

LISTA DE TABELAS

Tabela 1 – Resultados dos modelos LSTM com saída única para horizonte 1	52
Tabela 2 – Resultados dos modelos RNN tradicional com saída única para horizonte 1 .	53
Tabela 3 – Médias dos modelos LSTM com saída única para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.	54
Tabela 4 – Médias dos modelos RNN tradicional com saída única para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.	54
Tabela 5 – Desvio padrão dos modelos LSTM com saída única para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.	55
Tabela 6 – Desvio padrão dos modelos RNN tradicional com saída única para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.	55
Tabela 7 – Médias dos modelos LSTM com saída múltipla para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.	56
Tabela 8 – Médias dos modelos RNN tradicional com saída múltipla para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.	56
Tabela 9 – Desvio padrão dos modelos LSTM com saída múltipla para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.	57
Tabela 10 – Desvio padrão dos modelos RNN tradicional com saída múltipla para hori- zontes 1, 3, 5, 7, 9, 11, 13 e 15.	57

SUMÁRIO

1	INTRODUÇÃO	15
1.1	Problema de pesquisa	16
1.2	Objetivo principal	16
1.2.1	<i>Objetivos específicos</i>	16
1.3	Metodologia	16
1.4	Organização do Texto	17
2	CRIPTOMOEDAS E SÉRIES TEMPORAIS	19
2.1	Criptomoedas	19
2.1.1	<i>Bitcoin</i>	20
2.1.2	<i>Gráfico de candlesticks</i>	21
2.2	Séries Temporais	22
2.2.1	<i>Introdução às séries temporais</i>	22
2.2.2	<i>Elementos das séries temporais</i>	22
2.2.3	<i>Séries temporais do Bitcoin</i>	25
2.3	Predição de séries temporais	26
2.4	Visão geral de métodos de predição de séries temporais	29
2.4.1	<i>Previsores estatísticos</i>	29
2.4.2	<i>Previsores de aprendizado de máquina</i>	30
2.4.2.1	<i>Regressão Linear</i>	30
2.4.2.2	<i>Regressão de Support Vector Machine</i>	31
2.4.2.3	<i>Regressão de Árvore de Decisão</i>	31
2.4.3	<i>Previsores de aprendizado profundo</i>	32
2.4.3.1	<i>Redes Neurais Artificiais para previsão de séries temporais</i>	33
3	LONG SHORT-TERM MEMORY	35
3.1	Redes Neurais Artificiais	35
3.1.1	<i>Neurônio Artificial</i>	35
3.1.2	<i>Gradiente descendente e back-propagation</i>	37
3.1.3	<i>Redes Neurais Recorrentes</i>	39
3.1.4	<i>Redes Long Short-Term Memory</i>	40
3.1.5	<i>Métricas de tarefas de regressão adotadas</i>	42
4	REVISÃO DE TRABALHOS CORRELATOS	43

5	METODOLOGIA E RESULTADOS	47
5.1	Base de dados Bitcoin	47
5.2	Pré-processamento das séries temporais	47
5.2.1	<i>Normalização dos dados</i>	48
5.3	Arquitetura de LSTM para a predição de séries temporais	48
5.4	Resultados	49
5.4.1	<i>Hardware utilizado</i>	50
5.4.2	<i>Parametrização dos modelos</i>	50
5.4.3	<i>Modelo com saída única para diferentes horizontes</i>	52
5.4.4	<i>Modelo com múltiplas saídas</i>	54
6	CONCLUSÕES	59
	REFERÊNCIAS	61

INTRODUÇÃO

De acordo com [Nakamoto \(2009\)](#), criptomoedas (também chamadas de moedas eletrônicas) são definidas por uma cadeia de assinaturas digitais (*blockchain*), onde cada proprietário transfere a moeda para um destinatário, assinando digitalmente um *hash* da transação anterior e a chave pública do próximo. Este mecanismo de operação financeira é considerado seguro e tem a vantagem de não depender de políticas governamentais e nem mesmo privadas, por isso é um processo descentralizado. Conforme [Hileman e Rauchs \(2017\)](#), a quantidade de criptomoedas baseadas em blockchains, introduzidas por [Nakamoto \(2009\)](#) com o Bitcoin, está crescendo de forma exponencial nos últimos anos, influenciando os cenários comercial global e financeiro de forma significativa.

[Sovbetov \(2018\)](#) afirma que fatores relacionados ao mercado de criptomoedas, como beta de mercado, volume de negociação e volatilidade, parecem ser determinantes para criptomoedas como Bitcoin, Ethereum, Dash, Litecoin e Monero, tanto em curto prazo quanto em longo prazo. Os estudos do autor também mostram que existem muitos fatores que influenciam a tendência do preço da criptomoeda, o que a torna difícil de entender. Com isso, construir um mecanismo de previsão de preços de criptomoedas se torna importante para potenciais investidores e agências governamentais, uma vez que poderia diminuir os riscos (evitando possíveis prejuízos), tanto em ações de compra quanto ações de venda.

Diante destes cenários, estudos como os de [Awoke et al. \(2020\)](#), [Biswas et al. \(2021\)](#), [Indulkar \(2021\)](#), [Wu et al. \(2018\)](#) têm mostrado que é possível criar um modelo de previsão de séries temporais utilizando algoritmos de *deep learning*, como por exemplo, o *long short-term memory* (LSTM), que possui a característica de filtrar os dados relevantes, de forma a impedir os *vanishing and exploding gradients* e alcançar melhores resultados se comparado à redes neurais recorrentes (RNRs) tradicionais. Para o treinamento e teste do modelo, séries temporais de dados diários de criptomoedas podem ser coletadas através da API do CoinMarketCap¹, que é um site

¹ [<coinmarketcap.com>](https://coinmarketcap.com)

de monitoramento de preços de criptoativos.

Neste trabalho, foi investigado o desempenho da rede neural LSTM para a predição da cotação Real/Bitcoin com bases em séries temporais multivariáveis. Adicionalmente, para melhor validar o desempenho da rede neural LSTM foi comparado o desempenho desta com alguma outra rede neural recorrente.

1.1 Problema de pesquisa

O problema de pesquisa investigado neste trabalho consiste de: a partir de uma série temporal multivariável diária relacionada a cotação de Bitcoin (BTC) utilizando a moeda real (BRL), denominada de BTC/BRL, predizer a partir de uma janela temporal anterior w (o que é denominado de janela de predição), a taxa de conversão futura h de dias adiante, a qual é denominada horizonte de predição.

1.2 Objetivo principal

O objetivo principal deste trabalho foi criar um modelo que consiga fazer previsões da cotação BRL/BTC com base em dados passados dos preços e volumes diários.

1.2.1 Objetivos específicos

Os objetivos secundários foram:

- a) O estudo, implementação e experimentação de redes neurais para a predição de preços da criptomoeda Bitcoin.
- b) Desenvolvimento da rede neural LSTM e RNN tradicional para a predição da cotação BRL/BTC.
- c) Ajustes dos parâmetros dos modelos;
- d) Análise dos resultados envolvendo comparação dos modelos para diferentes horizontes de predição;

1.3 Metodologia

Os passos básicos para o desenvolvimento da trabalho foram:

1. Estudo da literatura sobre criptomoedas.
2. Obtenção de dados temporais de cotação de criptomoedas.

3. Normalização dos dados para melhor manuseio.
4. Implementação, treino, teste e ajuste de parâmetros dos modelos.
5. Investigação dos efeitos de diferentes tamanhos de janelas e horizontes de predição na rede LSTM e recorrente simples.
6. Investigação o desempenho da rede LSTM e recorrente simples para predição com horizonte simples e horizonte múltiplo. Predição com horizonte múltiplo refere a capacidade do modelo de prever múltiplas saídas referentes a diferentes horizontes de predição.
7. Estudo e implementação medidas de performance dos resultados.
8. Análise dos resultados e documentação destes juntamente com as técnicas usadas, na forma de artigo científico.
9. Comparar os resultados da rede LSTM com a rede neural recorrente simples.

1.4 Organização do Texto

O texto está organizado da seguinte maneira: primeiramente foi apresentado no capítulo de introdução, onde discorre-se sobre a necessidade predições da cotação de criptomoedas e introduz o programa de pesquisa. No [Capítulo 2](#) são apresentados conceitos de criptomoedas, séries temporais e uma breve revisão de modelos de predição de séries temporais. No [Capítulo 3](#) são apresentados os conceitos de redes neurais, com foco nas redes neurais LSTM e também as métricas de avaliação de desempenho de predição adotadas nesta pesquisa. No [Capítulo 4](#) são apresentados os trabalhos correlatos com foco na predição de séries temporais de cotação de criptomoedas. No [Capítulo 5](#) é apresentado a metodologia e os resultados obtidos. Por fim, o [Capítulo 6](#) apresenta as conclusões obtidas da pesquisa.

CRIPTOMOEDAS E SÉRIES TEMPORAIS

2.1 Criptomoedas

Segundo Nakamoto (2009), criptomoedas (também chamadas de moedas eletrônicas) podem ser definidas como uma cadeia de assinaturas digitais, onde cada proprietário transfere a moeda para um destinatário, assinando digitalmente um *hash* da transação anterior e a chave pública do próximo proprietário e adicionando-os ao final da moeda.

De acordo com a plataforma de mercados financeiros Investing¹, uma criptomoeda é um ativo digital criptografado que pode ser usado como meio de troca ou reserva de valor.

Conforme o banco digital Nubank², criptomoeda é o nome genérico para moedas digitais descentralizadas, criadas em uma rede *blockchain* a partir de sistemas avançados de criptografia que protegem as transações, suas informações e os dados de quem transaciona. A criptografia única de cada criptomoeda controla sua oferta monetária para evitar sua desvalorização, bem como o meio pelo qual as unidades podem ser criadas, armazenadas e transferidas com segurança.

Ainda segundo a plataforma Investing, as criptomoedas não são respaldadas por governos, bancos ou ativos subjacentes, como o ouro. Em vez disso, a moeda é controlada através de um sistema descentralizado de livros contábeis públicos, chamados *blockchains*. Esses livros contábeis são mantidos por particulares em computadores individuais, que formam o chamado sistema *peer-to-peer*, responsável por evitar que o valor do ativo seja afetado por qualquer entidade, ou seja, significa que as transações são gerenciadas por cada um dos usuários que fazem parte do sistema.

De acordo com o Nubank, de forma mais resumida, *blockchain* é uma cadeia de blocos onde cada um contém um arquivo e um *hash*, o que garante que as informações desse bloco de dados não foram violadas. Todo bloco criado contém sua *hash* e a do bloco anterior, criando uma

¹ <<https://br.investing.com/education/terms/criptomoeda-200226261>>

² <<https://blog.nubank.com.br/o-que-e-criptomoeda/>>

conexão entre os blocos.

O funcionamento pode ser resumido em:

1. Uma pessoa deseja enviar uma criptomoeda para outra;
2. O ativo é representado online como um bloco onde os detalhes estão armazenados;
3. O bloco é distribuído pela rede e cada máquina fica com uma cópia da transação em tempo real;
4. A rede verifica se o ativo é válido em questão de minutos;
5. Se aprovado, o bloco é adicionado a uma corrente de blocos e ganha um registro permanente na rede. Isso significa que ele não pode ser alterado;
6. A propriedade do ativo agora fica registrada na rede como sendo da pessoa que recebeu.

Toda essa movimentação é validada pela rede, mas ninguém sabe quem está fazendo a transação.

O Bitcoin e diversas outras criptomoedas utilizam a criptografia de chaves pública e privada. Nela, a rede da criptomoeda emite a todos os usuários uma chave privada, que é basicamente uma senha bem forte e, a partir dela, é gerada uma chave pública por *hashing*³. A chave pública pode ser passada para outras pessoas e é a única informação necessária para que um usuário envie unidades da criptomoeda para outro. Entretanto, para acessar esses fundos, a chave privada é necessária. Dessa forma, com as chaves pública e privada vinculadas, a rede sabe as unidades de uma criptomoeda que pertencem a um certo usuário (COINBASE, 2022).

As cotações das criptomoedas normalmente são expressas em um par, representado por uma moeda digital e uma moeda física. Um exemplo é a compra de Bitcoin (BTC) com a moeda brasileira, o real (BRL), que é expressa por BTC/BRL seguido de um valor, que mostra a quantidade necessária de reais para comprar uma unidade de Bitcoin.

2.1.1 Bitcoin

O Bitcoin foi inventado por Nakamoto (2009) com a publicação de seu trabalho denominado “Bitcoin: A Peer-to-Peer Electronic Cash System”. Ele criou um sistema de caixa eletrônico totalmente descentralizado que não depende de uma autoridade central para circulação de moeda e autenticação de transações.

A rede Bitcoin começou em 2009, com base em uma implementação de referência publicada por Nakamoto e desde então revisada por muitos outros programadores. A computação

³ Processamento de uma sequência de dados por um algoritmo. É praticamente impossível reverter esse processo e, por isso, ninguém consegue adivinhar uma chave privada a partir da chave pública.

distribuída que fornece segurança e resiliência para Bitcoin aumentou exponencialmente e agora excede a capacidade de processamento combinada dos principais supercomputadores do mundo (ANTONPOULOS, 2017).

O nome Satoshi Nakamoto é um pseudônimo e a identidade da pessoa ou pessoas por trás desta invenção é atualmente desconhecida. No entanto, nem Satoshi Nakamoto, nem ninguém, exerce controle sobre o sistema Bitcoin. A invenção em si é inovadora e já gerou novas ciências nas áreas de computação distribuída, economia e econometria (ANTONPOULOS, 2017).

2.1.2 Gráfico de *candlesticks*

O gráfico de *candlestick* é uma forma de mostrar o que ocorreu com o preço de um ativo⁴ em um determinado período de tempo. Eles foram desenvolvidos pelos japoneses no século XVIII, com o intuito de prever preços dos contratos de arroz (BIGTRENDS, 2015).

Nesse tipo de gráfico, a linha de preços é representada pelo eixo vertical, enquanto o eixo horizontal mostra a linha de tempo. O processo de montagem do gráfico consiste em representar cada pregão⁵ como uma “vela” posicionada verticalmente na escala gráfica de preços, na qual o “corpo da vela” dá destaque ao espaço ocupado entre o preço de abertura (preço pelo qual foi fechado o primeiro negócio do dia) e de fechamento (preço pelo qual foi fechado o último negócio do dia), e os preços máximo (preço mais alto pelo qual um negócio foi fechado naquele dia) e mínimo (preço mais baixo pelo qual um negócio foi fechado naquele dia) formam “pavios” acima e abaixo desse “corpo da vela”, se forem diferentes dos preços de abertura e fechamento. Esses “pavios” também são chamados de “sombas”. A cor em que se representa o corpo do *candle* é definida pelo movimento de preços que ocorreu no dia. Se o preço de fechamento for superior ao preço de abertura, o corpo do *candle* será representado na cor branca. Se o preço de fechamento for inferior ao preço de abertura, o corpo do *candle* será representado na cor preta. Assim, um *candle* branco representa um dia de alta no papel, e um *candle* preto, um dia de baixa (DEBASTIANI, 2007).

As cores branca e preta utilizadas para representar os *candles* de alta e baixa vêm da tradição religiosa oriental, em que os elementos Yin e Yang são simbolizados respectivamente pelo branco e o preto. Dessa maneira, não é obrigatório o uso dessas cores para tal representação, podendo-se usar cores como verde para alta e vermelho para baixa ou outras colorações.

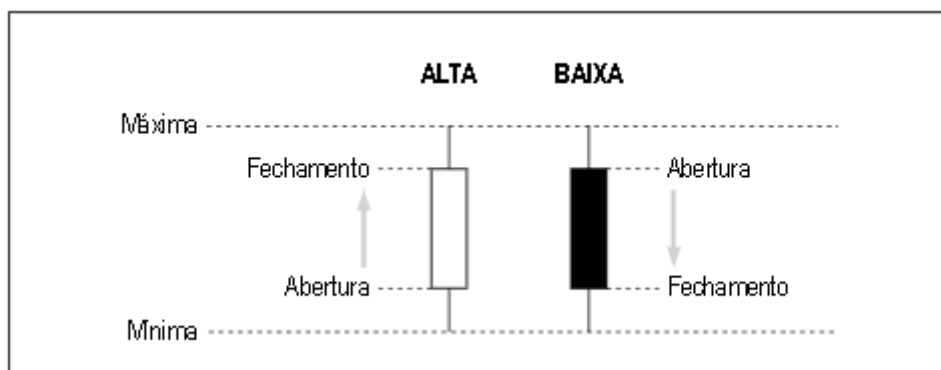
Um exemplo de *candle* de alta e de baixa pode ser visto na Figura 1.

Em função disso, o gráfico de *candlesticks* é bastante interessante para melhor visualização das cotações diárias das criptomoedas, sua flutuação e volatilidade.

⁴ O termo "ativo" é utilizado para representar algo que possui algum valor comercial, ou seja, um bem que pode ser negociado entre dois agentes.

⁵ Período em que a bolsa de valores está aberta para operações de compra e venda de investimentos.

Figura 1 – Formação dos candles.



Fonte: Adaptada de [Debastiani \(2007, página 18\)](#).

2.2 Séries Temporais

2.2.1 Introdução às séries temporais

Segundo [Brockwell e Davis \(2013\)](#), uma série temporal é um conjunto de observações $\{x_t, t = 0, \dots, n\}$, cada uma sendo registrada em um tempo específico t . Isso se resume basicamente a uma sequência de dados que foram reunidos em intervalos de tempo determinados durante um certo período, como dados diários, semanais, mensais, entre outros.

Os gráficos de séries temporais podem revelar padrões como aleatórios, tendências, mudanças de nível, períodos ou ciclos, observações incomuns ou uma combinação de padrões ([MONTGOMERY; JENNINGS; KULAHCI, 2015](#)). Um exemplo dessa diversidade de padrões pode ser percebido nos gráficos da [Figura 2](#) e da [Figura 3](#). Na [Figura 2](#) é mostrado um gráfico em que a produção de queijos azul e gorgonzola quadruplicou de 1950 a 1997, e a tendência linear tem uma inclinação positiva constante com variação aleatória de ano para ano. Já na [Figura 3](#) é revelada uma tendência geral de aumento para as remessas mensais de produtos de uma fabricante de bebidas dos EUA, com um padrão cíclico distinto que se repete a cada ano.

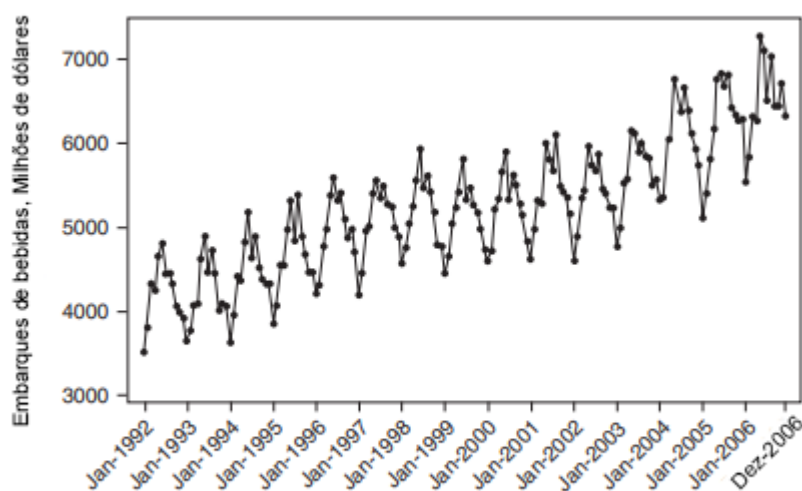
2.2.2 Elementos das séries temporais

Conforme [Chatfield \(2003\)](#), uma série temporal possui três componentes principais que a constitui: tendência, sazonalidade e resíduo. Esses componentes são descritos nos tópicos a seguir.

Tendência: O padrão denominado tendência possui um comportamento em um prazo mais alongado em que se torna possível constatar uma inclinação de alta, de queda ou de constância dos valores.

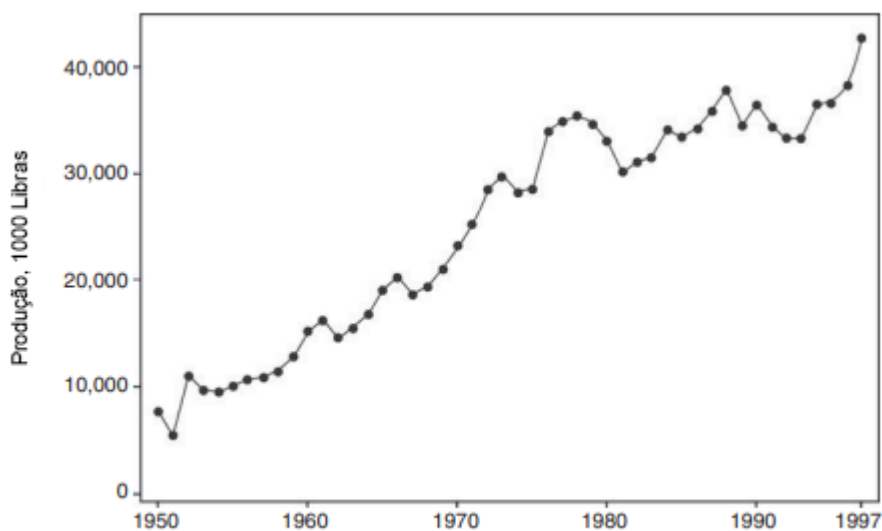
A tendência pode ser definida como “mudança de longo prazo no nível médio”. A dificuldade com esta definição é decidir o que se entende por “longo prazo”. Por exemplo, as

Figura 2 – Série temporal da produção anual dos EUA de queijos azul e gorgonzola.



Fonte: Adaptada de [Montgomery, Jennings e Kulahci \(2015, página 8\)](#).

Figura 3 – Série temporal dos embarques mensais de produtos da fabricante de bebidas dos EUA, não ajustados.



Fonte: Adaptada de [Montgomery, Jennings e Kulahci \(2015, página 9\)](#).

variáveis climáticas às vezes exibem variação cíclica ao longo de um período muito longo, como 50 anos. Se tivéssemos apenas dados de 20 anos, essa oscilação de longo prazo pareceria ser uma tendência, mas se dados de várias centenas de anos estivessem disponíveis, a oscilação de longo prazo seria visível. No entanto, no curto prazo, ainda pode ser mais significativo pensar nessa oscilação de longo prazo como uma tendência. Assim, ao falar de 'tendência', devemos levar em conta o número de observações disponíveis e fazer uma

avaliação subjetiva do que é 'longo prazo' na série temporal observada (CHATFIELD, 2003).

Sazonalidade: A sazonalidade se diz respeito a quando os eventos que acontecem durante um tempo determinado se repetem em um período igual, ou seja, fenômenos que ocorrem diariamente em uma certa hora, todos os dias, ou em um certo mês em todos os anos.

Muitas séries temporais, como números de vendas e leituras de temperatura, exibem variação que é anual no período (CHATFIELD, 2003). Um exemplo a ser citado é a variação no preço de hotéis de cidades turísticas, que podem ficar mais caros nos finais de semana, tendo um efeito sazonal.

Resíduo: Depois que a tendência e a sazonalidade são removidas de um conjunto de dados, ficamos com uma série de resíduos (ou ruídos), que podem ou não ser 'aleatórios' (CHATFIELD, 2003).

Para a visualização de cada um desses elementos, é utilizada a decomposição da série temporal, o que torna possível analisá-los individualmente. Esse procedimento implica em avaliar uma série temporal como uma junção de componentes, ou seja, tendência, sazonalidade, ruído e nível (valor médio da série). Com isso, se torna possível entender melhor a série temporal em questão e seus problemas relacionados a análise e previsão.

Os elementos de uma série temporal podem ser descritos como sistemáticos ou não sistemáticos. Os sistemáticos possuem consistência ou recorrência, além de poderem ser descritos e modelados, são eles: nível, tendência e sazonalidade. Já o ruído é um elemento não sistemático, ou seja, não pode ser modelado de forma direta (BROWNLEE, 2017).

Uma série temporal é uma combinação de tendência, sazonalidade, nível e ruído. Todas elas possuem nível e ruído, tendo tendência e sazonalidade como elementos opcionais (BROWNLEE, 2017). Além disso, podem possuir um modelo aditivo ou multiplicativo, fazendo parte da chamada Decomposição Clássica, que consiste em um modelo univariado que utiliza formulações matemáticas simples para separar a série nos quatro componentes principais citados (BOUZADA, 2012). Porém, nem sempre uma série temporal poderá ser decomposta por esse método, pois problemas reais podem ser bastante complexos e confusos, ou seja, pode haver componentes que são aditivos e outros multiplicativos, ou até mesmo uma mistura de elementos com comportamentos diferentes, entre outras possibilidades (BROWNLEE, 2017).

Modelo aditivo: Neste modelo, as mudanças ao longo do tempo são lineares. A tendência é linear e a sazonalidade possui mesma frequência e amplitude. O modelo aditivo é definido pela Equação 2.1 (BROWNLEE, 2017).

$$y(t) = \text{Nível} + \text{Tendência} + \text{Sazonalidade} + \text{Ruído} \quad (2.1)$$

Modelo multiplicativo: Neste modelo, as mudanças ao longo do tempo são não lineares, podendo ser quadrático por exemplo. A tendência é uma linha curva. A sazonalidade possui uma frequência e/ou amplitude crescente ou decrescente. O modelo multiplicativo é definido pela [Equação 2.2](#) ([BROWNLEE, 2017](#)).

$$y(t) = \text{Nível} * \text{Tendência} * \text{Sazonalidade} * \text{Ruído} \quad (2.2)$$

Esses modelos fornecem uma estrutura simplificada para a análise e exploração de dados, entretanto podem ser capazes ou não de separar os componentes de uma série temporal, o que leva a buscar métodos mais robustos para tal separação, dependendo do cenário.

Para ilustrar e exemplificar os componentes de uma série temporal, foi feita uma decomposição utilizando a série temporal do preço diário de fechamento do Bitcoin entre o período 29-03-2022 e 29-05-2022, descrita graficamente na [Figura 4](#). A princípio, não se consegue decompor esta série com o modelo aditivo ou multiplicativo, ou seja, não pode ser utilizado o método de decomposição clássica, então será usada a decomposição *Seasonal and Trend decomposition using LOESS* (STL) ([CLEVELAND et al., 1990](#)), onde LOESS vem do termo *locally estimated scatterplot smoothing*.

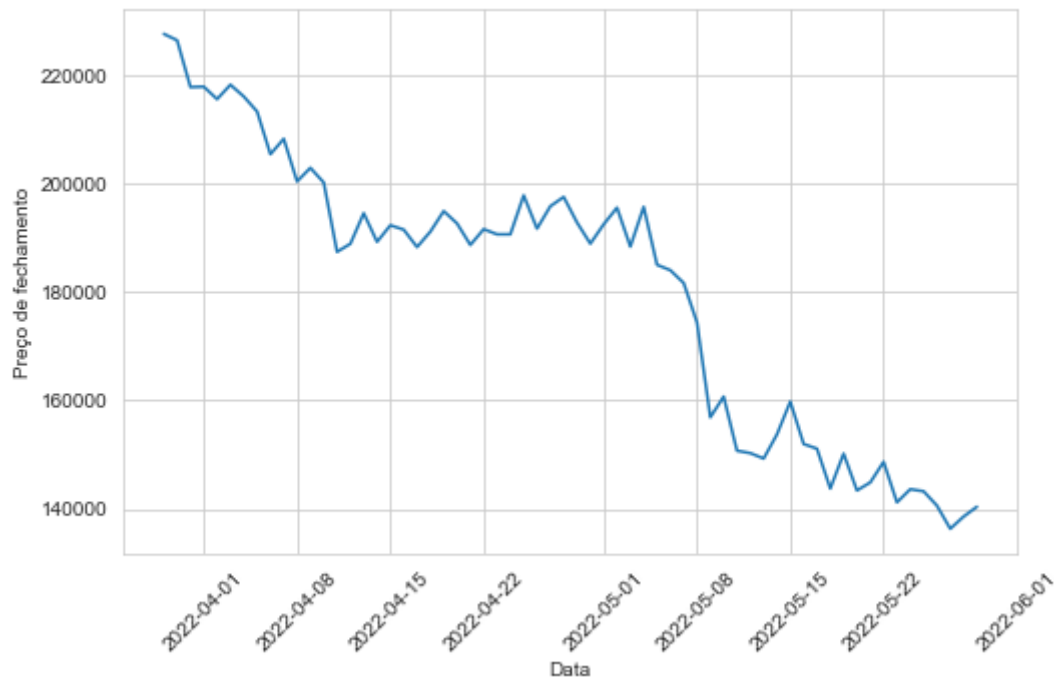
O STL é um método robusto de decomposição que possui algumas vantagens em relação à decomposição clássica, como a capacidade de lidar com qualquer tipo de sazonalidade e com valores discrepantes, de maneira que acontecimentos incomuns não prejudiquem estimativas dos componentes sazonais e ciclo de tendência. Na [Figura 5](#) pode-se ver os gráficos dos elementos da série após ser aplicada a decomposição STL, e dessa forma, eles podem ser analisados individualmente.

2.2.3 Séries temporais do Bitcoin

Quando se trata de definir um padrão para as séries temporais de criptomoedas, na verdade pode-se encontrar diversos padrões dependendo do período analisado. O Bitcoin, por exemplo, pode ser negociado em qualquer dia da semana, incluindo fins de semana e feriados, ou seja, a cada dia tem-se um valor de fechamento. Séries temporais desses valores podem conter tendências de alta, baixa e lateralidade (estabilidade), além de comportamentos aleatórios e mudanças de nível que podem acontecer devido à grande volatilidade que geralmente criptomoedas possuem. Além disso, vários fatores podem influenciar nas cotações, como crises, catástrofes, economia, entre outros.

Utilizando gráficos de *candlestick* é possível analisar os comportamentos de preço de forma mais completa, já que ele informa os preços de abertura, mínima, máxima e fechamento de um ativo, além de ser factível a análise de tendências, como em outros gráficos mais simples. Na [Figura 6](#) e [7](#), pode-se ver tendências de alta e de baixa respectivamente nos preços do Bitcoin sendo negociado com o Real (BRL). Já na [Figura 8](#), não tem-se um padrão claro de movimentação,

Figura 4 – Série temporal do preço de fechamento do Bitcoin.



Fonte: Desenvolvido pelo autor.

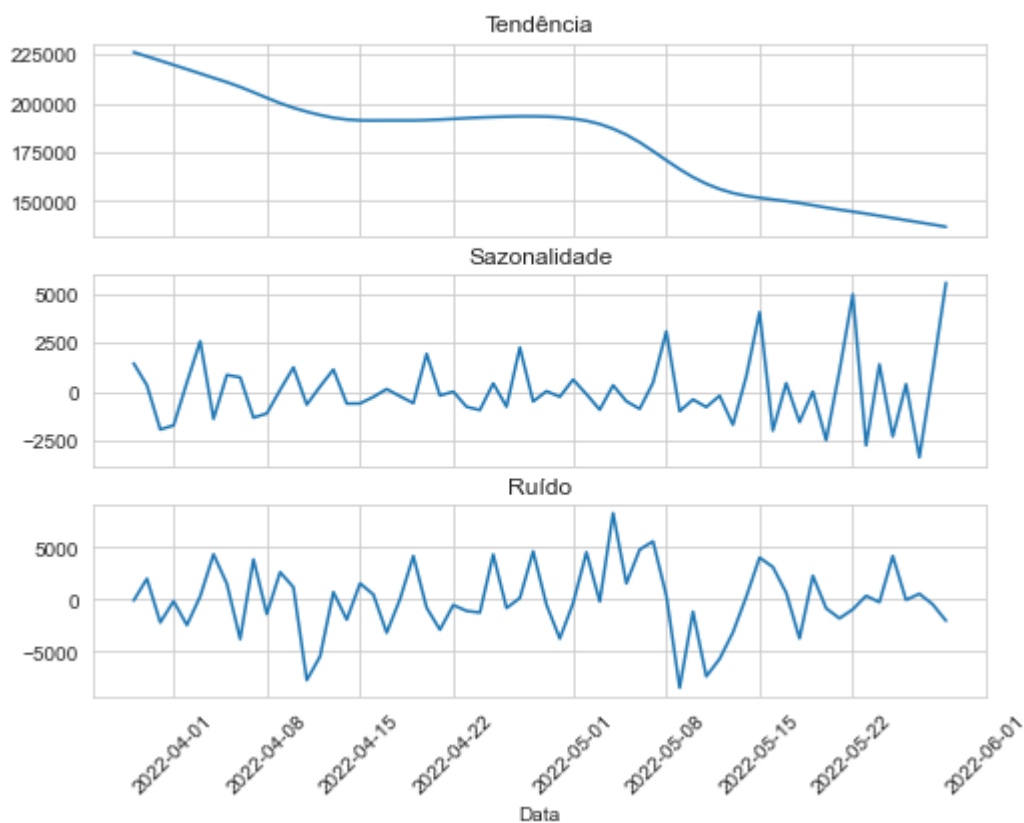
com várias subidas e quedas de preço. Vale ressaltar que os gráficos de *candlestick* são melhores para ver uma movimentação inteira de um ativo em um determinado intervalo de tempo, ou seja, para valores individuais se torna mais viável utilizar gráficos mais descomplicados, como o de linha.

2.3 Predição de séries temporais

A predição de séries temporais, segundo [Sorjamaa et al. \(2007\)](#), se diz respeito a ação de estimar o valor futuro de uma variável quantitativa com base somente nos seus dados históricos e respectivos inter-relacionamentos. Tal estimativa pode ser a curto, médio ou longo prazo. É importante salientar que o conceito de "prazo" pode depender do problema em questão, ou seja, para alguns problemas um mês pode ser longo prazo, já para outros é considerado como curto prazo e vice-versa. Um exemplo é o crescimento de uma empresa, onde uma previsão de lucro para um mês a um trimestre pode ser vista como curto prazo, porém pode ser vista como longo prazo para previsão meteorológica.

Para esse problema de predição, é necessário que haja uma modelagem da série temporal, com o intuito de criar um modelo que seja capaz de fazer projeções dos valores futuros. Os modelos podem ser univariados, que são baseados somente em uma série histórica, ou multivariados, que envolvem mais de uma série histórica ([SOUZA, 1981](#)). Além disso, há dois pontos importantes a serem definidos antes de construir um modelo de predição, são eles: janela

Figura 5 – Decomposição série temporal.



Fonte: Desenvolvido pelo autor.

Figura 6 – Tendência de alta no Bitcoin.



Fonte: Extraído da corretora de criptomoedas Binance.

deslizante e horizonte de predição.

De acordo com [Keogh e Lin \(2005\)](#), uma janela deslizante é dada pela Definição 1.

Figura 7 – Tendência de baixa no Bitcoin.



Fonte: Extraído da corretora de criptomoedas Binance.

Figura 8 – Padrão diferente de uma tendência no Bitcoin.



Fonte: Extraído da corretora de criptomoedas Binance.

Definição 1. Janela deslizante: Dada uma série temporal T de comprimento m e um valor definido pelo usuário de comprimento de subsequência w , uma matriz S de todas as subsequências possíveis pode ser construída deslizando uma janela sobre T e colocando a subsequência C_p na p -ésima linha de S . O tamanho da matriz S é $(m - w + 1)$ por w .

Em outras palavras, uma janela deslizante se resume a um campo de informação de tamanho fixo que desliza sobre todo o histórico de uma série temporal e é constantemente testada contra uma janela de previsão (horizonte de predição), tendo os dados mais antigos anteriores à janela descartados.

O horizonte de predição é o número de períodos futuros para os quais as previsões devem ser produzidas. O horizonte é muitas vezes ditado pela natureza do problema. Por exemplo, no planejamento da produção, as previsões da demanda do produto podem ser feitas mensalmente. Mercadorias concluídas para clientes ou instalações de estoque seria necessário previsões com até 3 meses de antecedência (MONTGOMERY; JENNINGS; KULAHCI, 2015).

2.4 Visão geral de métodos de predição de séries temporais

Há vários métodos de predição de séries temporais. Tais métodos podem utilizar certos modelos para a predição, como modelos estatísticos, de aprendizado de máquina e de aprendizado profundo.

2.4.1 Previsores estatísticos

Existem vários métodos estatísticos para a predição de séries temporais. Uma descrição abrangente de tais métodos pode ser encontrada em livros especializados como (BOX *et al.*,). Um dos previsores estatísticos bastante aplicado é modelo *AutoRegressive Integrated Moving Average* (ARIMA).

O modelo ARIMA, elaborado por Box, Jenkins e Reinsel (2008), é um modelo estatístico que envolve autoregressão (AR), integração (I) e médias móveis (*moving average* - MA). A autoregressão diz respeito à utilização de uma combinação linear de valores passados da própria variável de interesse para fazer sua previsão. A integração, por outro lado, está relacionada à introdução da diferença entre os valores passados e os atuais, que ajudam a garantir estacionariedade da série temporal. Por fim, as médias móveis utilizam valores anteriores da série temporal para prever o valor seguinte.

O modelo ARIMA é adequado quando a série temporal em questão é estacionária, ou seja, há uma constância de valores estatísticos como média e variância ao longo do período analisado (PARMEZAN, 2016).

O modelo é caracterizado por três termos: p, d, q. Primeiramente, para construir um modelo ARIMA é importante tornar a série temporal estacionária. Isso se deve ao fato de que o termo 'Auto Regressivo' no ARIMA significa que é um modelo de regressão linear que usa seus próprios atrasos como preditores. Os modelos de regressão linear funcionam melhor quando os preditores não estão correlacionados e são independentes uns dos outros. A partir disso, a abordagem mais comum para tornar uma série estacionária é diferenciá-la, ou seja, subtraia o valor anterior do valor atual. Às vezes, dependendo da complexidade da série, mais de uma diferenciação pode ser necessária. O valor de d, portanto, é o número mínimo de diferenciações necessárias para tornar a série estacionária. Se a série temporal já for estacionária, então $d = 0$. O

termo ‘p’ é a ordem do termo ‘Auto Regressivo’ (AR). Refere-se ao número de defasamentos da função a serem usados como preditores. Por fim, ‘q’ é a ordem do termo ‘Média Móvel’ (MA). Refere-se ao número de erros de previsão defasados que devem entrar no Modelo ARIMA (PRABHAKARAN, 2021).

Uma limitação normalmente apontada nos modelos de predição estatísticos é a dificuldade de tratar múltiplas variáveis que podem conter valores correlacionados, irrelevantes, ruídos diversos, entre outros aspectos (SIAMI-NAMINI; NAMIN, 2018). Essa limitação é amenizada por alguns modelos de aprendizado de máquina, em especial por redes neurais profundas (LIM; ZOHREN, 2021).

2.4.2 Previsores de aprendizado de máquina

Segundo Parmezan (2016), previsores baseados em aprendizado de máquina procuram descrever as propriedades dos dados sem antes conhecer a distribuição deles. Dessa maneira, se tornam mais simples de serem ajustados e conseguem ter bom desempenho em séries temporais complexas. Além disso, métodos de aprendizado de máquina possuem a vantagem de fazerem ajustes automáticos para os dados em questão, já que são modelos orientados a dados.

Os algoritmos de aprendizado de máquina são geralmente divididos em três classes: Aprendizado Supervisionado, Aprendizado Não-Supervisionado e Aprendizado por Reforço. Este trabalho em questão irá focar na classe supervisionada, onde os algoritmos têm a característica de usar a saída correta para induzir funções que mapiem um grupo de características de entrada para a saída correta ou alvo (LUXBURG; SCHÖLKOPF, 2011).

Além disso, dentro dos modelos supervisionados, o foco principal será no tipo de modelagem de regressão, onde o algoritmo tem a tarefa de prever valores contínuos (números reais). Alguns exemplos de algoritmos de regressão serão descritos resumidamente a seguir.

2.4.2.1 Regressão Linear

Ng (2003) explica que para realizar o aprendizado supervisionado utilizando regressão linear, deve-se decidir como será a representação do cálculo. Inicialmente pode-se aproximar y como uma função linear de x (Equação 2.3), considerando que x pode ser um vetor multidimensional.

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 \dots \quad (2.3)$$

De acordo com a Equação 2.3, θ_i são os parâmetros que fazem o mapeamento linear de X para Y. Para simplificar a notação, pode-se colocar x_0 igual a 1 (termo de interceptação).

A Equação 2.4 mostra o resultado dessa modificação, onde θ e x ao lado direito são vetores e n é o número de variáveis de entrada (desconsiderando o x_0).

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x \quad (2.4)$$

Dessa forma, dado um conjunto de treinamento, para aprender os parâmetros θ busca-se aproximar $h(x)$ de y . Formalizando isto, obtém-se uma função que estima, para cada valor de θ , quão perto $h(x^{(i)})$ está do seu y correspondente. A partir disso, tem-se uma função de custo que pode ser vista na [Equação 2.5](#).

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad (2.5)$$

Assim, o objetivo é ajustar os parâmetros θ de forma a minimizar a [Equação 2.5](#). Para ajustar os parâmetros θ pode-se aplicar variados métodos de minimização de funções, contudo um dos métodos mais usado é o gradiente descendente ([NG, 2003](#)).

2.4.2.2 Regressão de Support Vector Machine

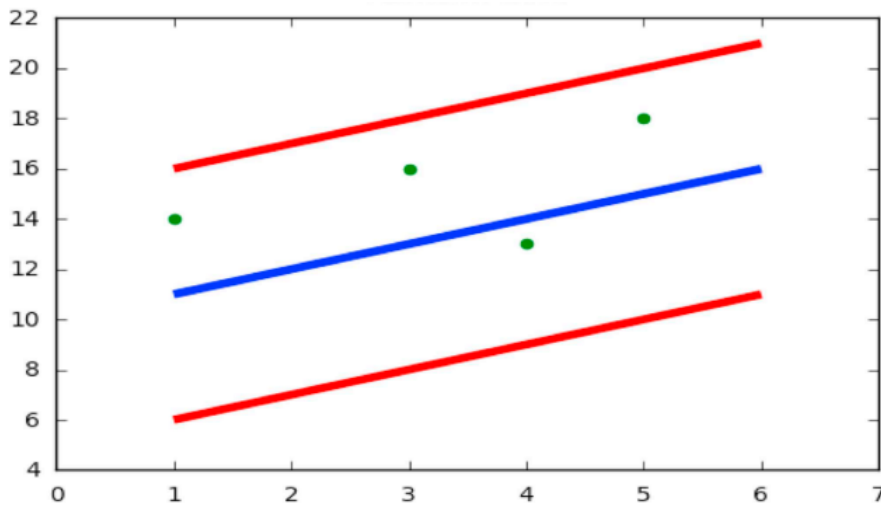
De acordo com [Sakthivel e Susila \(2021\)](#), Máquinas de Vetor de Suporte (*Support Vector Machines* - SVMs) são modelos de aprendizado supervisionado com algoritmos de aprendizado associados que analisam e investigam a análise de regressão de padrões com base em classificações. Em *Support Vector Machine Regression* (SVR), distingue-se um hiperplano com aresta mais extrema de tal forma que o maior número de foco de informação está dentro dessa aresta. SVMs se caracterizam pelo uso de kernels, pela esparsidade da solução e pelo controle de capacidade obtido atuando na margem, ou no número de vetores de suporte, entre outros. A capacidade do sistema é controlada por parâmetros que não dependem da dimensionalidade do espaço de características.

Como o algoritmo SVM opera nativamente em atributos numéricos, ele usa uma normalização z-score em atributos numéricos. Na regressão, os algoritmos SVM usam a função de margem de perda para resolver problemas de regressão. Em vez de limitar a taxa de erro como na regressão linear direta, tenta-se ajustar o erro dentro de uma borda específica. O objetivo em SVR é considerar fundamentalmente os focos que estão dentro da borda. A linha de melhor ajuste é o hiperplano que possui o número mais extremo de focos. No algoritmo SVM, a margem “c” é sempre considerada uma margem de variação. Na [Figura 9](#), as linhas vermelhas (superior e inferior) representam os padrões com a margem “c” na margem dos eixos positivo e negativo.

2.4.2.3 Regressão de Árvore de Decisão

Segundo [Sakthivel e Susila \(2021\)](#), as árvores de decisão podem ser utilizadas para classificação e regressão. Nas árvores de decisão, em cada nível, é necessário distinguir a propriedade de divisão.

Figura 9 – SVR com margem



Fonte: Adaptada de Sakthivel e Susila (2021).

Uma árvore de decisão é trabalhada distribuindo as informações em subconjuntos contendo exemplos com qualidades comparáveis (homogêneas). O desvio padrão é utilizado para calcular a homogeneidade de um exemplo matemático. Caso o exemplo matemático seja totalmente homogêneo, seu desvio padrão é zero (SAKTHIVEL; SUSILA, 2021). A seguir está descrito de forma resumida quais são os passos para encontrar o nó de divisão:

1. Calcule o desvio padrão da variável alvo usando a Equação 2.6.

$$\sigma = \sqrt{\left(\frac{1}{N}\right) \sum_{i=1}^N (x_i - \mu)^2} \quad (2.6)$$

2. Divida o conjunto de dados em vários atributos e calcule o desvio padrão para cada ramo (desvio padrão para alvo e indicador). Este valor é deduzido do desvio padrão antes da divisão. O resultado é a diminuição do desvio padrão (Equação 2.7).

$$\text{SDR}(T, X) = S(T) - S(T, X) \quad (2.7)$$

3. O atributo com a maior diminuição do desvio padrão é escolhido como o nó de partição.
4. O conjunto de dados é particionado dependendo das estimativas da característica escolhida. Essa interação é executada recursivamente nos ramos não-folha até que todas as informações sejam tratadas.

2.4.3 Previsores de aprendizado profundo

O aprendizado profundo possui uma inspiração biológica, onde cientistas da computação e analistas quantitativos ficavam se perguntando se a maneira de construir máquinas inteligentes

era imitar cérebros, com suas redes de neurônios disparando em resposta a certos gatilhos. Além disso, houve inspiração vinda do campo matemático e do aprendizado de máquina, já que o crescimento das capacidades de computação juntamente com a ascensão desse subcampo da ciência da computação, mostrou que com dados e parâmetros suficientes, sistemas complicados podem ser modelados e previstos (NIELSEN, 2019).

Ainda de acordo com Nielsen (2019), o aprendizado profundo levou tais ideias ainda mais longe ao criar redes descritas por milhões de parâmetros, treinados em grandes conjuntos de dados e com uma base teórica de que uma rede neural deve ser capaz de representar uma função arbitrária não linear com alto grau de precisão.

2.4.3.1 Redes Neurais Artificiais para previsão de séries temporais

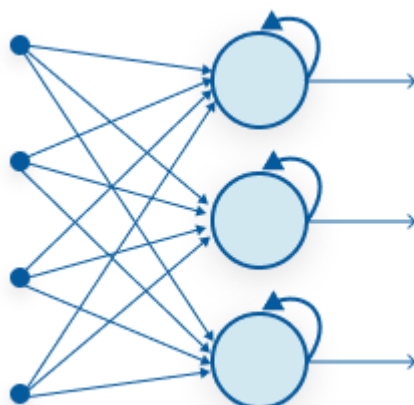
Antes de descrever as Redes Neurais Artificiais (RNA), é importante discutir um pouco sobre uma diferença entre as redes superficiais e as profundas. Kim e Gofman (2018) diz que desde a criação das RNA, elas desenvolveram várias formas e tipos, incluindo as categorias superficial (ou rasa) e profunda. Tais termos se referem ao número de camadas em sua estrutura. As redes neurais superficiais referem-se àquelas que possuem um pequeno número de camadas, geralmente consideradas como tendo uma única camada oculta, já as profundas, referem-se àquelas que possuem várias camadas ocultas. Ambos os tipos de redes executam certas tarefas melhor do que a outra, e selecionar a profundidade de rede correta é importante para criar um modelo de sucesso.

Ainda de acordo com Kim e Gofman (2018), redes rasas são mais adequadas para tarefas de aprendizado que lidam com dados com baixa dimensionalidade e um número relativamente pequeno de características. Por outro lado, as redes neurais profundas se destacam em tarefas muito complexas que possuem grandes dados de entrada e alta dimensionalidade. Dessa forma, é importante que se saiba que não é porque o algoritmo é uma rede neural, que ele necessariamente utiliza aprendizado profundo.

Conforme Haykin (2009), as redes neurais artificiais são modelos matemáticos inspirados no funcionamento dos neurônios biológicos. As RNA com múltiplas camadas de neurônios possuem a propriedade de suas funções de aproximação conseguirem mapear qualquer função não-linear sem qualquer estimativa antecedente relacionada aos dados. Devido suas características adaptativas, elas são eficazes no mapeamento da dinâmica das séries temporais não estacionárias e, nos últimos anos, têm colaborado para o desenvolvimento de métodos de predição mais precisos para séries temporais (KAUSHIK; GIRI, 2020).

Existem também as Redes Neurais Recorrentes (RNR) que seguem o princípio das RNAs, porém com uma arquitetura recorrente, isto é, possuem uma conexão recorrente entre neurônios de processamento (Figura 10). Essa recorrência garante que as informações sequenciais sejam capturadas nos dados de entrada. Além disso, elas compartilham os parâmetros em diferentes etapas de tempo. Isso resulta em menos parâmetros para treinar e diminui o custo computacional.

Figura 10 – Exemplo de rede neural recorrente.



Fonte: Adaptada de [Pai \(2020\)](#).

Segundo [Kaushik e Giri \(2020\)](#), se tratando de previsão de séries temporais, a RNR mais usada é a *Long Short-Term Memory* (LSTM), que possui o diferencial de conseguir identificar a aleatoriedade dos dados melhor que uma RNA simples, além de ser capaz de lidar com o erro de decaimento da retropropagação (problema presente em RNRs clássicas) devido ter blocos de memória que auxiliam nisso, e isso a torna superior para predição de séries temporais com longas dependências temporais.

LONG SHORT-TERM MEMORY

3.1 Redes Neurais Artificiais

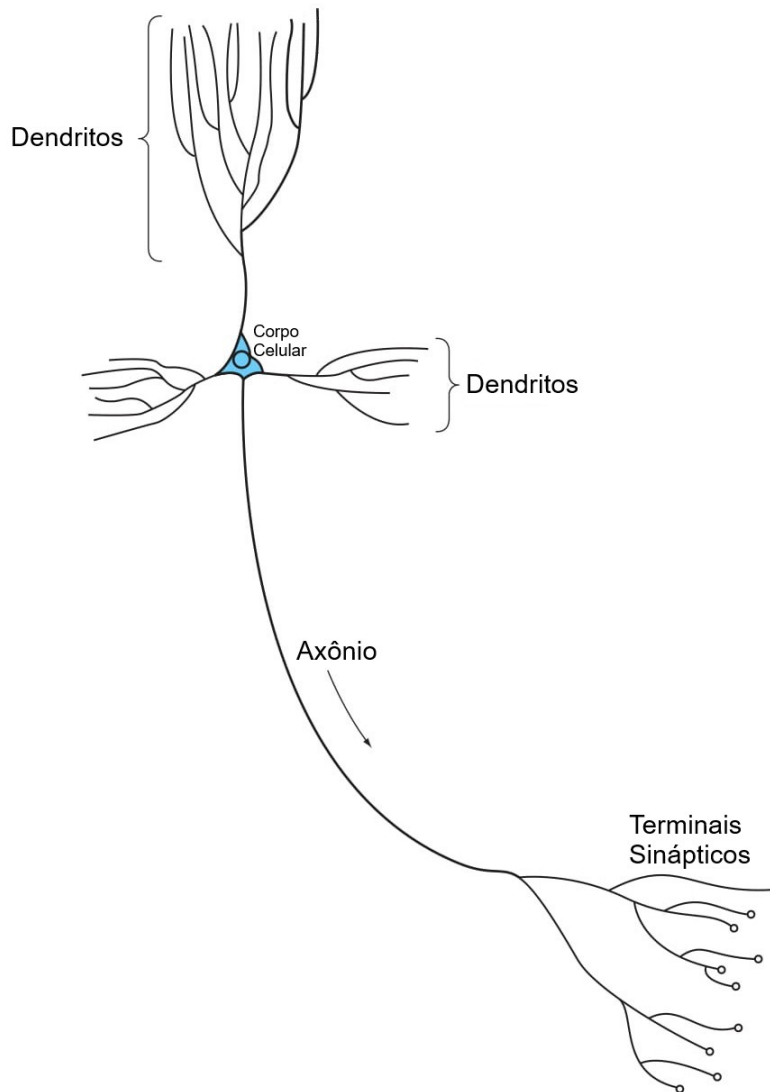
Dando continuidade ao que foi explicado na [subseção 2.4.3.1](#), redes neurais artificiais (RNA) são técnicas de aprendizado de máquina que procuram simular o aprendizado de organismos biológicos. No sistema nervoso humano, existem os neurônios ([Figura 11](#)), que são células com capacidade de se comunicarem. Tal comunicação acontece através dos dendritos, que são responsáveis pela recepção de estímulos; axônios que garantem a condução de um impulso nervoso; e sinapses, região onde há a comunicação entre os neurônios. Os estímulos recebidos influenciam na força das conexões, e é isto que define o aprendizado para os organismos em questão. As redes neurais artificiais se assemelham a esse mecanismo através de elementos computacionais que imitam os neurônios e se conectam a outros através de pesos, que simbolizam as forças das sinapses.

3.1.1 Neurônio Artificial

Na [Figura 12](#) tem-se uma representação de um neurônio artificial, que tenta imitar um neurônio biológico. Os sinais elétricos que foram passados pelos axônios de outros neurônios caracterizam as entradas. Os pesos que simulam as sinapses podem ter seu valor aumentado ou diminuído antes de ser somado com outros em um corpo celular. Quando acontece a soma da entrada ponderada ($w_{k1} * x_1 + w_{k2} * x_2 + \dots + w_{km} * x_m + 1 * b_k$), chega-se a um resultado que será passado para a função de ativação. A chamada função de ativação modela a forma como o neurônio responde ao nível de excitação, limitando e definindo a saída da rede neural ([HAYKIN, 2009](#)).

Um neurônio também possui o *bias*, que é um parâmetro que é aprendido pela rede e possui o objetivo de aumentar (se for positivo) ou diminuir (se for negativo) o valor de entrada da função de ativação. Dessa forma, em termos matemáticos, um neurônio de índice k pode ser

Figura 11 – Ilustração de um neurônio.



Fonte: Adaptada de Haykin (2009).

representado pela equação:

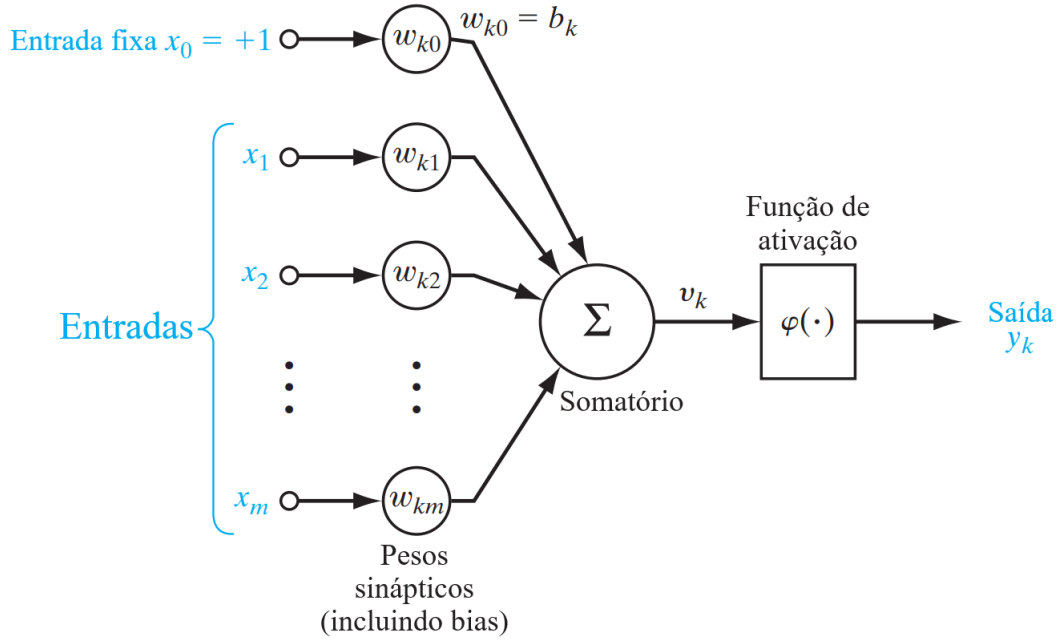
$$v_k = \sum_{j=0}^m w_{kj} x_j \quad (3.1)$$

onde $x_0 = 1$ e w_{k0} é o bias, que dessa maneira é aprendido pela rede.

Logo após isso, o campo local induzido v_k passa por uma função de ativação ϕ , para delimitar a saída em um dado intervalo (geralmente valores entre 0 e 1 ou -1 e 1), ocasionando na saída y_k (HAYKIN, 2009).

Há diversas funções de ativação de neurônios, dentre elas, tem-se por exemplo a função *sigmoide*, tangente hiperbólica (*tanh*) e *threshold*. A *sigmoide* é dada pela Equação 3.2 e é uma função que possui um equilíbrio entre comportamento linear e não linear, além de ser crescente e

Figura 12 – Ilustração de um neurônio artificial.



Fonte: Adaptada de Haykin (2009).

seu valor de retorno ser restrito entre 0 e 1. A *tanh*, dada pela Equação 3.3 possui comportamento parecido, porém seus valores de retorno variam entre -1 e 1. Já a função de *threshold*, dada pela Equação 3.4, retorna o valor 1 se receber um valor maior ou igual a 0, e retorna o valor 0 se receber um valor menor que 0.

$$\varphi(v) = \frac{1}{1 + \exp(-av)} \quad (3.2)$$

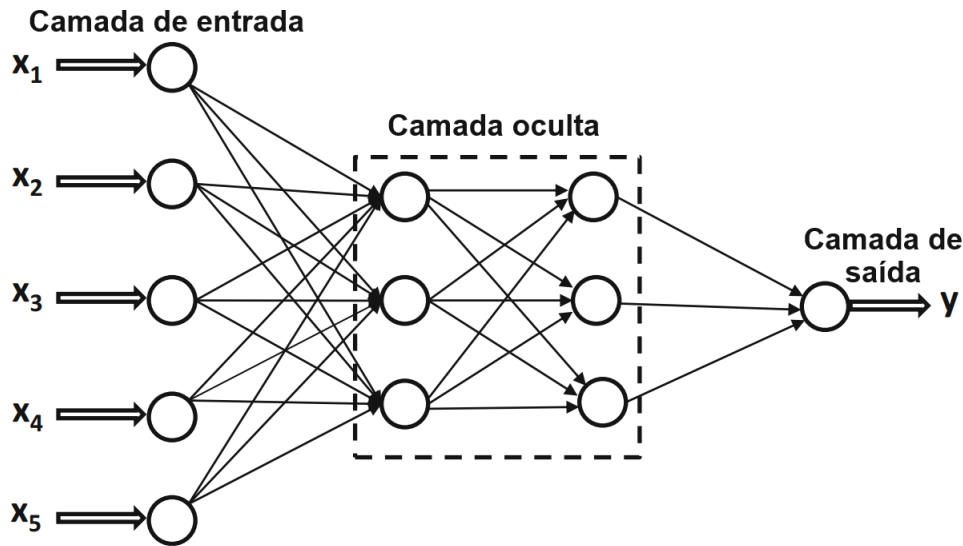
$$\tanh(v) = \frac{e^v - e^{-v}}{e^v + e^{-v}} \quad (3.3)$$

$$\varphi(v) = \begin{cases} 1 & \text{se } v \geq 0 \\ 0 & \text{se } v < 0 \end{cases} \quad (3.4)$$

Uma RNA normalmente possui sua organização em camadas, onde cada camada abrange um ou mais neurônios com saídas ligadas e conectadas com neurônios da próxima camada. Dentre as arquiteturas existentes, a mais comum é a chamada *feedforward* (Figura 13), que consiste em uma rede com fluxo de informação unidirecional, com sentido da camada de entrada para a de saída, sem ciclos.

3.1.2 Gradiente descendente e back-propagation

O *back-propagation* (ou retropropagação) é um algoritmo de aprendizado supervisionado desenvolvido para a otimização de redes neurais. Esse algoritmo funciona com base no conceito

Figura 13 – Arquitetura *feedforward*.

Fonte: Adaptada de Aggarwal (2018).

de gradiente descendente, que possui sua implementação baseada no gradiente da função objetivo (função que minimiza o erro dado um conjunto de treinamento), que tem sua definição dada através do conjunto das derivadas parciais da função objetivo relacionada aos parâmetros de otimização (PEREIRA; CUSTODIO, 2021).

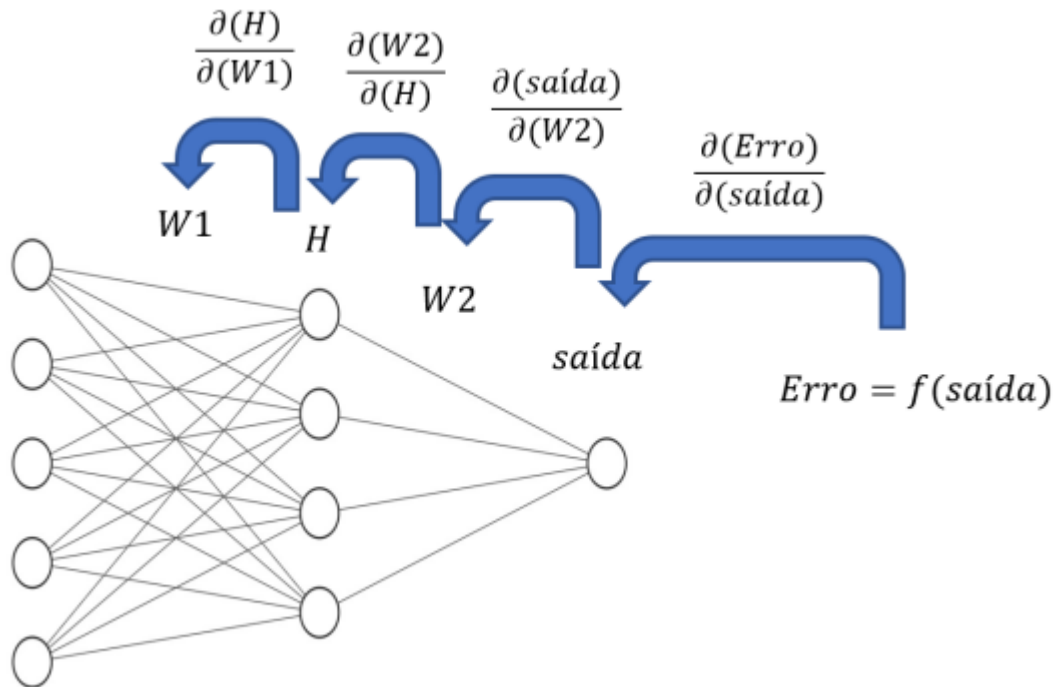
Ainda segundo Pereira e Custodio (2021), o gradiente descendente possui a função de orientar a direção e tamanho dos “passos” a serem tomados pelos parâmetros. Para minimização da função objetivo, o passo é em direção oposta ao gradiente, caracterizando um gradiente descendente, já para maximização, o passo é em direção ao gradiente, caracterizando um gradiente ascendente. O gradiente é calculado partindo da última camada indo em sentido à primeira, e dessa forma, o gradiente parcial da camada seguinte é utilizado para se computar o gradiente da camada anterior através da regra da cadeia. Esse processo pode ser visto na Figura 14, onde são computados os gradientes da função de erro em relação a cada uma das camadas utilizando *backpropagation*. O gradiente é usado em seguida para estabelecer os novos valores de cada um dos pesos de forma iterativa.

A retropropagação termina quando o gradiente da função objetivo é definido em relação aos parâmetros da camada mais distante da função (ou seja, a primeira camada $W1$ da rede). Dessa forma, é definido para a rede em questão os gradientes da função erro em relação a cada um dos conjuntos de parâmetros utilizando a regra da cadeia das derivadas (Equação 3.5 e Equação 3.6).

$$\frac{\partial(\text{Erro})}{\partial(W2)} = \frac{\partial(\text{Erro})}{\partial(\text{saida})} * \frac{\partial(\text{saida})}{\partial(W2)} \quad (3.5)$$

$$\frac{\partial(\text{Erro})}{\partial(W1)} = \frac{\partial(\text{Erro})}{\partial(\text{saída})} * \frac{\partial(\text{saída})}{\partial(W2)} * \frac{\partial(W2)}{\partial(H)} * \frac{\partial(H)}{\partial(W1)} \quad (3.6)$$

Figura 14 – Cálculo do gradiente e *back-propagation*.



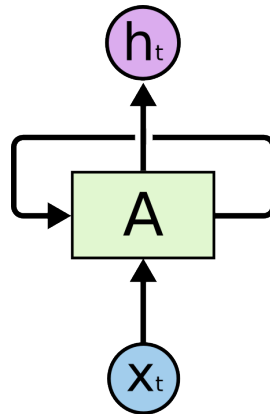
Fonte: [Pereira e Custodio \(2021\)](#).

Resumidamente, tendo como objetivo a otimização dos pesos da rede de maneira a encontrar uma conexão entre as entradas e saídas, o funcionamento do *back-propagation* se dá através de uma distribuição das entradas como sinais para os neurônios da camada de entrada. Estes sinais são propagados através das camadas da rede, podendo ser aumentados, diminuídos e agregados até que sejam gerados os sinais de saída da rede. Após isso, faz-se uma comparação das saídas de rede com as saídas já conhecidas que foram passadas, e isso gera um valor de erro. O erro é passado para o algoritmo de aprendizagem que, a partir de um algoritmo de otimização, faz o ajuste dos pesos. Todo esse processo é repetido até que o erro tenha diminuído até o mínimo possível, conforme as amostras do conjunto de treinamento.

3.1.3 Redes Neurais Recorrentes

Em consequência da demanda de prever dados que possuem dependências temporais (séries temporais e frases de texto, por exemplo), houve uma adaptação das redes neurais na qual se possui a alternativa de incluir ciclos internos de sinal em sua estrutura. Assim, se houver ao menos um ciclo de sinal (*feedback loop*) na rede neural, ela é chamada recorrente ([Figura 15](#)) ([HAYKIN, 2009](#)).

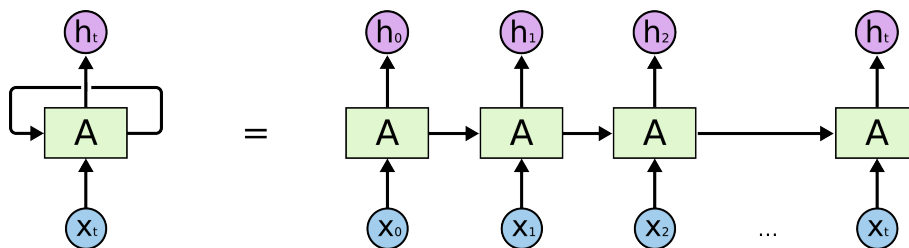
Figura 15 – Estrutura de uma rede neural recorrente.



Fonte: Olah (2015).

As redes neurais recorrentes (RNRs) são constituídas por *loops* que fazem com que as informações de saída de uma estrutura de processamento seja alimentada de volta a esta construção. Desta forma, pode-se considerar que RNRs são como diversas cópias de uma estrutura, onde cada uma delas passa certas informações para a cópia seguinte. Essa construção de ciclo (Figura 16) permite que informações anteriores alimentadas à rede sejam retidas, desempenhando um papel semelhante a uma memória. Vale ressaltar que essa estrutura é um pedaço da rede, podendo assim conter múltiplos neurônios e camadas.

Figura 16 – RNR estendida



Fonte: Olah (2015).

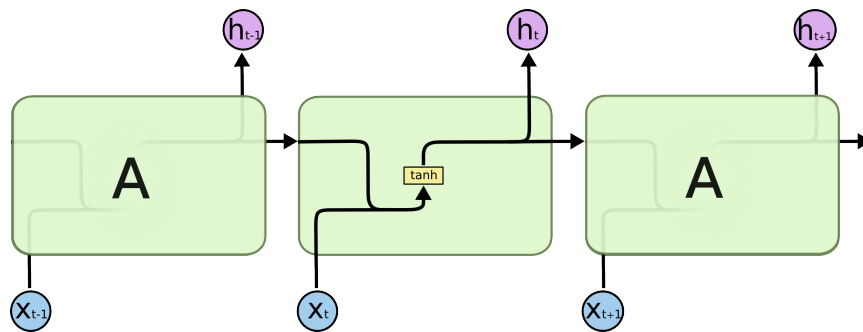
Apesar da importância dos ciclos para desempenhar o papel de memória, Hochreiter e Schmidhuber (1997) abordam o problema das dependências de longo prazo dessas redes, em que elas não conseguem prever um dado armazenado na memória de longo prazo. Porém, elas conseguem prever de forma mais precisa a partir de informações recentes. As redes *long short-term memory* (LSTM) foram desenvolvidas para tratar este problema, inserindo o conceito de portões, e com isso conseguem guardar informações por longos períodos de tempo.

3.1.4 Redes Long Short-Term Memory

As LSTMs foram propostas por Hochreiter e Schmidhuber (1997) especificamente para escapar do problema de dependência de longo prazo. A diferença entre as RNR tradicionais e LSTM está basicamente na estrutura. A RNR tradicional possui uma estrutura de ciclo, que

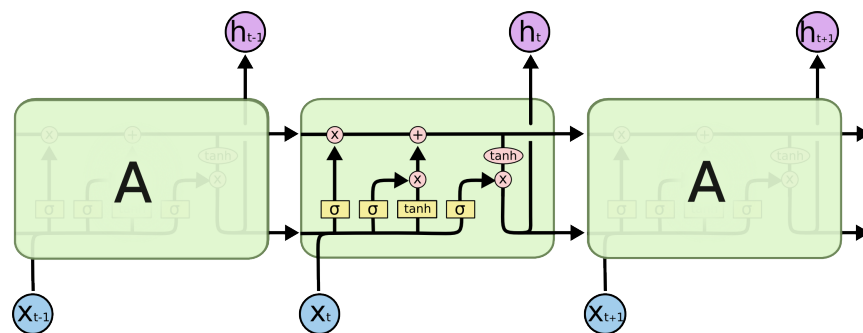
pode ser visualizada como uma cadeia de módulos repetitivos ao longo do tempo (Figura 17). Considerando uma RNR padrão, as unidades (neurônios) têm uma composição bastante elementar com uma simples função de ativação na saída destes, que alimenta ou retroalimenta neurônios deste módulo. Já numa LSTM, a estrutura de cadeia de informação temporal também existe, porém o módulo de repetição é bem diferente, pois a memória ao invés de ser implementada por ciclos de sinal podendo envolver múltiplos neurônios e camadas da rede, é implementada (retida e liberada) através de um conceito de portões (*gates*), que controla uma célula de memória (Figura 18).

Figura 17 – RNR - Estrutura de cadeia e módulo de repetição



Fonte: Olah (2015).

Figura 18 – LSTM - Estrutura de cadeia e módulo de repetição



Fonte: Olah (2015).

Uma célula LSTM, que é o elemento básico de uma rede LSTM, contém quatro elementos, sendo três portões (*Forget gate*, *Input gate* e *Output gate*) e o estado da célula (*Cell state*), que tem o papel de guardar a memória que é manipulada.

O *Forget gate* decide a quantidade de memória que vai permanecer e a quantidade que vai ser descartada. Ele analisa entrada x_t e a saída do cálculo anterior h_{t-1} e utiliza uma função sigmoide para obter um valor entre 0 e 1. O valor 0 possui o significado de esquecer completamente, e o valor 1 significa lembrar de tudo. Este processo é relacionado com cada informação da memória, e após isso, o estado da memória é atualizado.

O *Input gate* determina quais informações vão entrar na memória. Ele analisa a entrada x_t e a saída do cálculo anterior h_{t-1} , usa uma função sigmoide, e uma função tangente hiperbólica

(*tanh*) para normalizar os valores de entrada. O valor após passar pela função sigmoide é multiplicado pelo resultado da função *tanh*. De acordo com isso, o cálculo gera as informações que devem ser guardadas na memória. Por fim se soma o valor final ao estado da célula para incrementar as informações na memória.

Por fim, o *Output gate* decide quais informações relevantes das etapas anteriores são necessárias para serem usadas como saída da célula. Ele analisa entrada x_t e a saída do cálculo anterior h_{t-1} e aplica a função sigmoide. Após isso, o resultado é multiplicado pelo estado da célula aplicado em uma função *tanh*. Este processo resulta no processamento final do bloco de memória.

Estas células de memória LSTM podem ser organizadas de modo a ter múltiplas camadas e múltiplas células por camadas. Organizações complexas com muitas comadas são normalmente referidas como arquiteturas de *deep learning*.

3.1.5 Métricas de tarefas de regressão adotadas

As métricas de avaliação adotadas para este trabalho foram a raiz do erro quadrático médio (root mean squared error – RMSE) (Equação 3.7) e o erro percentual médio absoluto (mean absolute percentage error – MAPE) (Equação 3.8).

- Root mean squared error (RMSE):

$$RMSE = \sqrt{MSE} \quad (3.7)$$

- Mean absolute percentage error (MAPE):

$$MAPE = 100 \times \frac{1}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{y_i} \quad (3.8)$$

O RMSE é a raiz do erro médio quadrático da diferença entre valor real e predição. É uma medida similar ao desvio padrão, e seu valor é interpretado como uma medida do desvio médio entre observado e predito.

O MAPE mede o erro em porcentagem e seu cálculo é dado como a média do erro percentual, indicando a acurácia do erro em porcentagem. Isso quer dizer que se o resultado do cálculo foi de 5%, então o erro médio do modelo é de 5%, dessa forma, quanto menor o MAPE, melhor é o ajuste.

REVISÃO DE TRABALHOS CORRELATOS

O modelo LSTM convencional possui três fases: avaliação do período de atraso; treinamento e previsão; modelo e avaliação. Na busca por criar uma nova estrutura de previsão para prever o preço diário do Bitcoin, [Wu et al. \(2018\)](#) utilizaram dois modelos LSTM diferentes (modelo LSTM convencional e um modelo híbrido LSTM com modelo AR(2) – LSTM-AR(2)). Esse estudo usou dados diários de preço e volume de transações do bitcoin, entre 01/01/2018 e 28/07/2018. Na proposta híbrida apresentada, temos um passo a mais que o modelo convencional chamado "Teste ACF, PACF", que são construções estatísticas teóricas e é realizado na primeira fase do modelo. As métricas de erro utilizadas foram o *mean squared error* (MSE), *root mean square error* (RMSE), *mean absolute percentage error* (MAPE), e *mean absolute error* (MAE). Ambos os modelos conseguiram resultados de previsão próximos aos valores reais, porém os quatro valores do índice de avaliação do segundo modelo são menores que os valores do primeiro modelo, representando assim, um desempenho superior.

[Awoke et al. \(2020\)](#) propuseram a utilização de modelos de aprendizado profundo, realizando especialmente uma comparação entre *long short-term memory* (LSTM) e *gated recurrent unit* (GRU), para lidar com a volatilidade do preço do Bitcoin e tentar conseguir uma precisão alta na previsão dos preços da criptomoeda. Os dados coletados são do intervalo de 01/01/2014 a 20/02/2018. Além disso, o *dataset* possui sete atributos, como preço de abertura, preço máximo, preço mínimo e preço de fechamento. Para validar o desempenho dos modelos, foram utilizados o *mean absolute percentage error* (MAPE) e *root mean square error* (RMSE) como medidas de desempenho, realizadas para previsões de 1, 3, 5, 7 e 15 dias à frente e janelas de tamanho 1, 5, 7, 12 e 15 respectivamente. Através disso, foi constatado que o GRU é mais adequado para prever dados de séries temporais de maior volatilidade.

[Iqbal et al. \(2021\)](#) buscaram fazer análises de séries temporais baseadas em aprendizado de máquina. Para isso, foram utilizados os modelos ARIMA, FBProphet e XG Boosting. O dataset utilizado inclui oito atributos relacionados aos dados históricos do Bitcoin, ou seja, é

multivariável. No pré-processamento, foram feitos principalmente a normalização para alguns atributos e remoção de *outliers*. Através disso, o modelo ARIMA se saiu melhor que os outros modelos em termos das seguintes medidas de desempenho: *root mean square error* (RMSE), *mean absolute error* (MAE) e R^2 .

No trabalho de [Indulkar \(2021\)](#) foram feitas análises de várias criptomoedas, dentre elas Bitcoin (BTC), Ethereum (ETH), Chainlink (LINK), Bitcoin Cash (BTC) e XRP através dos algoritmos *long short-term memory* (LSTM) e Fbprophet. O dataset utilizado possui preços diários das moedas e abrange seis atributos: preço de abertura, fechamento, máxima, mínima diários e métricas relacionadas ao volume. Os dados coletados são do ano de 2018 ao ano de 2020. A métrica de avaliação utilizada foi a *mean absolute error* (MAE), e foi gerada para todas as criptomoedas. O menor valor de erro foi gerado para a criptomoeda Bitcoin que foi de 0,01867, seguido pelo Bitcoin Cash que foi de 0,02632. Além disso, o Fbprophet que é utilizado para previsão e compreensão da mudança de tendência com base na sazonalidade, foi utilizado para análises individuais de cada criptomoeda, com um tamanho de horizonte de predição de 15 dias. Os resultados variaram de moeda para moeda, onde algumas se ajustaram melhor que as outras.

[Biswas et al. \(2021\)](#) utiliza múltiplas variáveis para fazer a previsão de preços das criptomoedas Litecoin (LTC) e Monero (XMR), como capitalização do mercado de ações, volume e distribuição, levando em consideração preços de abertura, fechamento, máxima e mínima. Os dados da Litecoin são de 24 de agosto de 2016 a 23 de fevereiro de 2020 e da Monero são de 30 de janeiro de 2015 a 23 de fevereiro de 2020. O modelo proposto é uma mistura de Gated Recurrent Unit (GRU) e Long Short-Term Memory (LSTM) e o pré-processamento utilizado foi a norma Min-Max. A métrica de avaliação utilizada foi a *root mean square error* (RMSE), para ciclos de um dia, três dias e sete dias. A conclusão foi de que o modelo proposto se mostrou melhor que o modelo LSTM tradicional.

O trabalho de [Derbentsev et al. \(2020\)](#) é dedicado à previsão de séries temporais de criptomoedas utilizando aprendizado de máquina. Foram aplicados os algoritmos *Random Forest* e *Gradient Boosting Machine*. Já na parte de dados, foram usados os preços de fechamento diários e volumes de negociação das criptomoedas Bitcoin (BTC), Ethereum (ETH) e Ripple (XRP). O conjunto de dados corresponde a 01/01/2015 a 31/12/2019 para BTC e XRP, e de 07/08/2015 a 31/12/2019 para ETH. Ambos os modelos tiveram a mesma ordem de precisão, se mostrando eficientes para a previsão de séries temporais das criptomoedas em questão, contudo é investigado somente a predição para o dia seguinte. As métricas de avaliação adotadas foram o Erro Percentual Absoluto Médio (MAPE) e Erro da Raiz Quadrática Média (RMSE).

[Gao, He e Kuruoglu \(2021\)](#) propõe em seu estudo uma abordagem híbrida que utiliza uma combinação da Heterocedasticidade Condicional Auto-Regressiva (GARCH) com Long-Short Term Memory (LSTM). Os dados utilizados foram os preços de fechamento e volume diários do Bitcoin entre o período de 01/01/2015 e 02/09/2021. Como pré-processamento foi utilizada uma

transformação logarítmica para estabilizar a variância da série temporal. Tal modelo mostrou desempenhos preditivos semelhantes utilizando as métricas de avaliação MSE, MAE e RMSE. A conclusão do estudo foi que a combinação de modelos paramétricos como GARCH com redes neurais profundas pode trazer melhores resultados na previsão de preços de criptomoedas, especialmente no caso de dados altamente voláteis ou quando sequências curtas de dados estão disponíveis.

Li e Dai (2020) também utiliza uma abordagem híbrida em seu trabalho, porém com *Convolutional Neural Network* (CNN) e Long-Short Term Memory (LSTM). Os dados de treino consistem nas informações históricas do Bitcoin (BTC) e variáveis macroeconômicas e a atenção do investidor em relação à criptomoeda. A CNN é usada para extrair as características que têm grande influência no preço do Bitcoin no conjunto de dados e, em seguida, a LSTM é usada para previsão dos preços. Como pré-processamento foi usada uma padronização com o intuito de evitar influência das diferentes dimensões das variáveis. As métricas de avaliação utilizadas foram o Erro Médio Absoluto (MAE), Erro Quadrático Médio da Raiz (RMSE) e Erro Percentual Absoluto Médio (MAPE). Como conclusão tem-se que a rede neural híbrida CNN-LSTM tem um bom desempenho na previsão de Bitcoin e é mais adequada para a previsão da criptomoeda. O modelo é testado para prever a volatilidade BTC ao longo de 10 dias, sendo investigado o número de camadas da rede LSTM e o tamanho de janela deslizante; os melhores resultados obtidos com três camadas LSTM e janela deslizante de tamanho dez.

O estudo de Jakubowicz e Abdelfattah (2021) teve como propósito prever a direção esperada do Bitcoin (BTC) para a hora seguinte. Ao usar dados de negociação com intervalo de uma hora e criar classes discretas de níveis de mudanças horárias, cinco modelos diferentes de Machine Learning foram treinados e testados, sendo eles: Regressão Logística, *Support Vector Machine*, *Random Forest*, KNN e *Decision Tree*. O conjunto de dados utilizado neste experimento contém o preço de abertura, máxima, mínima e fechamento da criptomoeda em intervalos de 1 minuto, entre os anos de 2016 e 2021. No pré-processamento foram criados rótulos em que através de uma porcentagem de alteração a cada hora, se dava uma categoria para aquele intervalo. Os rótulos são: *High Loss*, *Medium Loss*, *Low Loss*, *Minimal Loss*, *No Change*, *Minimal Gain*, *Low Gain*, *Medium Gain* e *High Gain*. Como resultado teve-se que, com exceção do KNN, uma precisão da validação cruzada variando de 96-100% foi alcançada. Os melhores resultados foram obtidos por *Random Forest* e *Decision Tree*.

Levando em consideração que a volatilidade dos preços das criptomoedas é uma das suas principais desvantagens, o artigo de Fleischer *et al.* (2022) teve o intuito de explorar uma análise de séries temporais usando aprendizado profundo para estudar e entender tal comportamento volátil. Os dados utilizados possuem intervalo de um dia (ou seja, são dados diários) e o estudo foca em prever o fechamento do próximo dia através de uma rede Long-Short Term Memory (LSTM). As criptomoedas utilizadas no estudo foram Electro-Optical System (EOS), Bitcoin, Ethereum e Dogecoin. Os resultados obtidos com o modelo LSTM foram comparados com o

modelo ARIMA, e o LSTM produziu uma Raiz quadrada do erro-médio (RMSE) menor, ou seja, se mostrou melhor na comparação, porém teve um tempo de execução maior. Segundo os autores, o modelo obteve um erro médio bastante pequeno, o que significa que o valor previsto é bastante próximo do preço real.

METODOLOGIA E RESULTADOS

Neste capítulo é descrito como foi obtida a série histórica de preços diários do Bitcoin, o pré-processamento dos dados para viabilizar a predição, a definição do modelo LSTM, as métricas usadas para avaliação, e por fim os resultados obtidos juntamente com uma análise destes.

5.1 Base de dados Bitcoin

A base de dados com informações históricas de transações do Bitcoin (BTC) utilizando a moeda real (BRL) foi obtida através do *site* Cryptodatadownload ¹, que contém séries históricas de várias criptomoedas com diversas periodicidades e intervalos de tempo. A série temporal obtida é de periodicidade diária, datada entre 27-12-2020 e 08-08-2022, e possui as seguintes variáveis/colunas: *unix*, *date*, *symbol*, *open*, *high*, *low*, *close*, Volume BTC, Volume BRL e *tradecount*.

Os dados utilizados para este estudo consiste das variáveis *open* (preço de abertura), *high* (máxima do dia), *low* (mínima do dia), *close* (fechamento do dia), Volume BTC e Volume BRL, que representam o volume das transações em Bitcoin e em real, respectivamente.

5.2 Pré-processamento das séries temporais

Segundo (BATISTA, 2003), como os dados são o meio utilizado para a extração de conhecimento dentro do aprendizado de máquina, a qualidade e o pré-processamento adequado destes implica diretamente no desempenho que um modelo vai ter.

Há diversos fatores presentes em um determinado conjunto de dados que pode prejudicar o desempenho de uma rede neural. No caso dos dados de Bitcoin usados, não há problemas como

¹ <https://www.cryptodatadownload.com/data/>

dados faltantes nas séries temporais, porém se tem diferenças grandes na ordem de grandeza das variáveis que compõem os dados, e isso pode ser resolvido através da normalização dos dados.

5.2.1 Normalização dos dados

BATISTA (2003) define normalização como a transformação dos valores originais de um conjunto de dados para valores em um determinado intervalo, normalmente entre -1 e 1 ou 0 e 1. Tal transformação possibilita que a diferença de escala entre as múltiplas variáveis que compõem as séries temporais não afetem a técnica de predição utilizada. Nos resultados iniciais do estudo foi utilizada a normalização de cada variável no intervalo entre 0 e 1, através das seguintes fórmulas aplicadas sequencialmente:

$$std = \frac{X - \min(X)}{\max(X) - \min(X)} \quad (5.1)$$

$$scaled = std * (lim_{sup} - lim_{inf}) + lim_{inf} \quad (5.2)$$

onde X é um vetor de valores correspondentes a uma variável, $\min(X)$ e $\max(X)$ são respectivamente o menor e o maior valor de X , e lim_{sup} e lim_{inf} são os valores limites para a normalização, com lim_{inf} sendo o menor valor e lim_{sup} o maior valor. Nos experimentos lim_{inf} foi setado com o valor 0 e lim_{sup} foi setado com o valor 1.

5.3 Arquitetura de LSTM para a predição de séries temporais

Para este trabalho foi elaborado um modelo LSTM de múltiplas camadas (Figura 19), sendo uma camada de 1024 neurônios e uma de 512. Após as duas camadas LSTM é aplicada uma única camada densa, com a quantidade de neurônios igual a quantidade de dados a serem previstos (a quantidade de dados a serem previsto varia de experimento para experimento). A quantidade de neurônios da camada de entrada (que não fazem processamento) também varia conforme o experimento, sendo dada pelo tamanho de janela temporal w utilizado, multiplicado pela quantidade de variáveis (seis variáveis, conforme a base de dados utilizada).

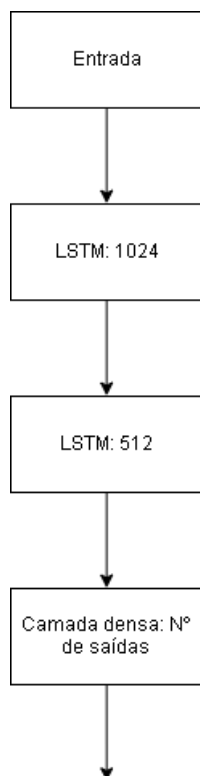
A função de ativação escolhida para a saída da rede neural de cada modelo foi a *linear*, já que é uma função que não altera a saída de um neurônio. A *tanh* foi utilizada como função de ativação das camadas LSTM.

Por fim, na saída da rede, o valor resultante é desnormalizado antes do cálculo de erro, sendo comparado com o valor real de cotação do conjunto de teste. Para a desnormalização usa-se a seguinte Equação:

$$\hat{out} = out * (\max(Y) - \min(Y)) + \min(Y) \quad (5.3)$$

onde *out* é a predição normalizada gerada pela rede neural, $\max(Y)$ e $\min(Y)$, são respectivamente os valores máximo e mínimo da variável alvo da predição (no caso a cotação BRL/BTC) sobre o conjunto de treinamento. Assim, *ôut* é a predição desnormalizada que é comparada com o respectivo valor real da cotação dada no conjunto de teste, para a mensuração do erro do modelo.

Figura 19 – Modelo LSTM.



Fonte: Desenvolvido pelo autor.

5.4 Resultados

Foram feitos três experimentos utilizando a arquitetura LSTM especificada na [seção 5.3](#). O primeiro deles visa a experimentação de parâmetros de forma a decidir a parametrização mais promissora do modelo para os demais experimentos. Para isso, o modelo é configurado para prever uma saída única, que se trata da predição da cotação BRL/BTC para o dia seguinte. O segundo experimento visa analisar o desempenho deste modelo de única saída, quando se aumenta o horizonte de predição para 3, 5, 7, 9, 11, 13 e 15 dias. O terceiro experimento consiste da alteração do modelo para prever em uma única execução, múltiplas saídas, correspondentes à cotação BRL/BTC para os horizontes de predição (1, 3, 5, 7, 9, 11, 13 e 15 dias); neste caso o modelo passa a ter oito neurônios na última camada.

Além disso, os mesmos experimentos foram feitos para redes neurais recorrentes tradicionais, utilizando a mesma estrutura que os modelos LSTM, para efeito de comparação entre RNN tradicional e LSTM.

Em todos os experimentos a base de dados de Bitcoin foi separada de forma sequencial conforme o tempo (data) em treinamento e teste, sendo 80% dos registros iniciais usados para treinamento e 20%, que correspondem aos registros restantes usados para teste. Também, em todos os experimentos foram utilizadas 50 épocas de treinamento, que foram executadas utilizando uma taxa de aprendizado de valor 0,001. O objetivo é conseguir prever o preço de fechamento do Bitcoin, ou seja a cotação BRL/BTC no final do dia.

5.4.1 Hardware utilizado

Todos os experimentos foram feitos utilizando um computador com um processador Ryzen 5 5600G e 16GB de memória ram DDR4 3200mhz. Os processos foram feitos através da CPU, não sendo utilizada nenhuma GPU.

5.4.2 Parametrização dos modelos

O primeiro experimento teve como objetivo definir os parâmetros mais promissores dos modelos (LSTM e RNN). Para isso foram experimentados os valores 16, 32 e 64 para tamanho de lote (*batch*) de treinamento; 3, 7 e 15 para tamanho de janela deslizando; e *rmsprop* e *adam* como otimizadores. Ambos os otimizadores são baseados no cálculo de gradiente descendente descrito no [Capítulo 3](#).

O *rmsprop* ([HINTON G.; SWERSKY, 2012](#)) faz a atualização dos pesos usando uma suavização do tipo média móvel para calcular o termo v_n em função dos gradientes anteriores:

$$v_n = \gamma v_{n-1} + (1 - \gamma) \Delta_n^\omega \quad (5.4)$$

onde γ é um parâmetro que controla o peso do gradiente atual em relação aos gradientes anteriores. A regra de atualização dos pesos se torna:

$$\omega^{(l)}(n+1) = \omega^{(l)}(n) + \frac{\eta}{\sqrt{v_n + \epsilon}} \Delta_n^\omega \quad (5.5)$$

onde η é a taxa de aprendizagem, Δ_n^ω corresponde o valor de gradiente, ϵ é um pequeno valor constante, para evitar divisão por zero. Para γ foi utilizado o valor 0.9 e para ϵ o valor 10^{-7} .

O otimizador *adam*, definido em ([KINGMA, 2014](#)), mantém controle dois vetores de atualização com base no gradiente:

$$m_n = \beta_1 m_{n-1} + (1 - \beta_1) \Delta_n^\omega, \quad v_n = \beta_2 v_{n-1} + (1 - \beta_2) \left(\Delta_n^\omega \right)^2, \quad (5.6)$$

onde β_1 e β_2 são parâmetros que controlam a atualização dos vetores em função do gradiente momentâneo. Para evitar problemas numéricos esses vetores são normalizados da seguinte forma:

$$\hat{m}_n = \frac{m_n}{1 - \beta_1}, \quad \hat{v}_n = \frac{v_n}{1 - \beta_2} \quad (5.7)$$

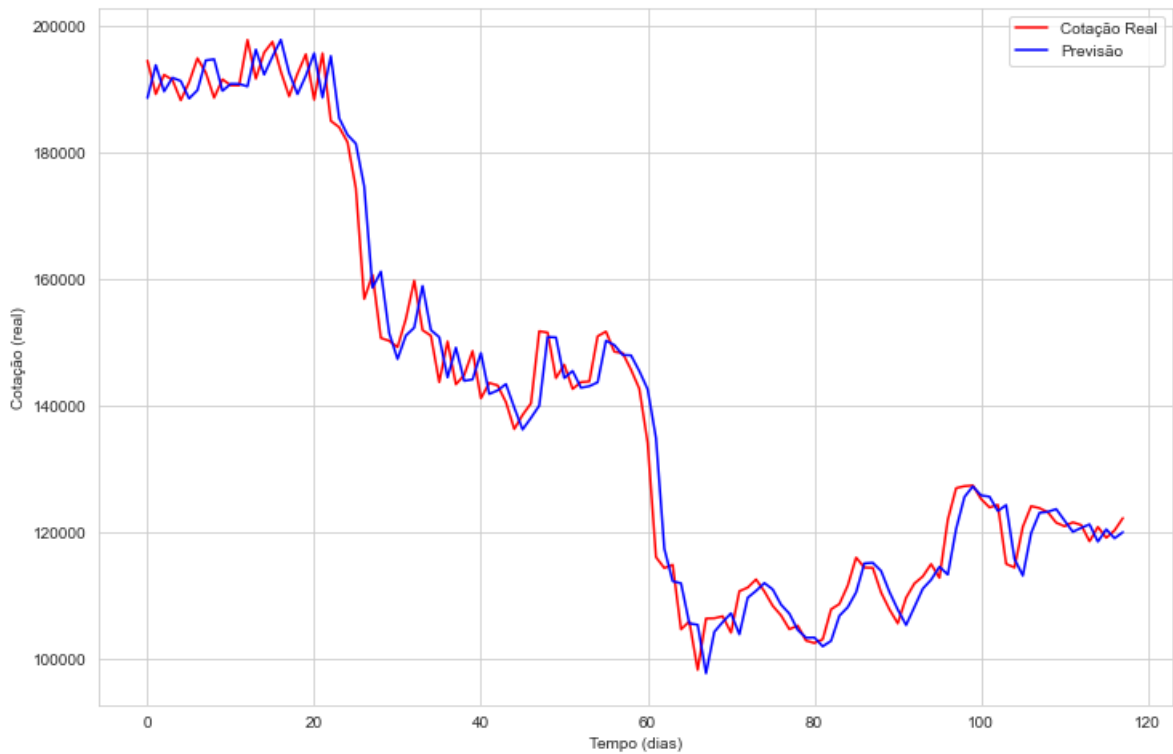
A equação de atualização dos pesos de *adam* é dada por:

$$\omega_n + 1 = \omega_{n-1} - \frac{\eta}{\hat{v}_n + \varepsilon} \hat{m}_n \quad (5.8)$$

onde η é uma taxa de aprendizagem e ε é um fator para evitar divisão por zero. Nos experimentos com o otimizador *adam* foram utilizados os seguintes valores de parâmetros: $\beta_1 = 0.9$, $\beta_2 = 0.999$, e $\varepsilon = 10^{-7}$

Na [Tabela 1](#) é mostrado o erro médio absoluto (MAE), o erro percentual médio absoluto (MAPE) e a raiz quadrada do erro médio (RMSE) do modelo LSTM para cada uma das configurações, ordenado do menor ao maior MAE. De acordo com os testes realizados, vê-se que a melhor configuração foi o modelo com tamanho de *batch* igual a 16, otimizador *adam* e tamanho de janela igual a 3, que alcançou um MAPE de aproximadamente 2,58%. O gráfico de comparação entre preço real e previsão do modelo LSTM com melhor configuração pode ser visto na [Figura 20](#).

Figura 20 – Modelo LSTM - Preço real vs previsão



Fonte: Desenvolvido pelo autor.

Ainda sobre a [Tabela 1](#), pode se notar que os modelos com o otimizador *adam* produziram menor taxa de erro do que os modelos com o *RMSPProp*. Também se nota que tamanho de lote igual a 16 se destacou entre os tamanho testados. Já em relação ao tamanho de janela, é observado que não há um padrão muito claro. Adicionalmente, quanto menor o tamanho de janela, significa que o modelo gera a previsão usando menos dados, pois a quantidade dados de entrada da rede é dada pelo tamanho de janela multiplicado pelo número que variáveis (este último fixo).

Tabela 1 – Resultados dos modelos LSTM com saída única para horizonte 1

Tamanho de lote	Otimizador	Tamanho de janela	MAE	MAPE	RMSE	Tempo de execução (s)
16	adam	3	3562.264963	0.025830	4838.878512	79.255878
16	adam	7	3670.724775	0.026795	5048.383636	152.651244
32	adam	3	3722.770723	0.027353	5089.601753	47.331855
16	adam	15	3750.131488	0.027546	5039.361691	307.016288
32	adam	7	3741.781912	0.027610	5043.085533	95.786722
16	rmsprop	15	3895.775556	0.028038	5352.294848	354.202212
64	rmsprop	15	4206.803430	0.031314	5624.246991	125.468110
64	adam	7	4179.205376	0.031518	5476.914766	58.456244
64	adam	15	4441.096796	0.033296	5862.256001	120.596262
32	adam	15	4446.565877	0.033303	5863.366008	202.265591
64	adam	3	4611.539526	0.035035	5997.362157	28.373571
32	rmsprop	7	4708.248212	0.035312	6043.450660	112.397270
16	rmsprop	3	4915.712063	0.037912	6295.985881	112.964758
64	rmsprop	3	6071.258011	0.042225	7840.336933	36.368211
64	rmsprop	7	6211.685050	0.047998	7690.952830	67.106068
32	rmsprop	15	6593.290188	0.051499	7896.688857	218.834930
32	rmsprop	3	6493.311176	0.051555	7910.407060	62.079630
16	rmsprop	7	12857.283766	0.094431	13900.280030	192.772917

Fonte: Desenvolvido pelo autor.

Na [Tabela 2](#) são mostrados os dados e resultados dos experimentos feitos com a RNN tradicional. É possível notar que a maior diferença foi que o melhor modelo (segundo o MAE) teve as configurações semelhantes às do melhor modelo LSTM, com exceção do tamanho de lote, onde na RNN o tamanho 32 foi o melhor e o tamanho 16 o segundo melhor. Outro ponto interessante é que o RMSE do segundo melhor modelo é menor que o do menor primeiro melhor modelo. Além disso, a diferença entre o MAE de ambos é muito baixa. O gráfico de comparação entre preço real e previsão do modelo RNN tradicional com melhor configuração pode ser visto na [Figura 21](#).

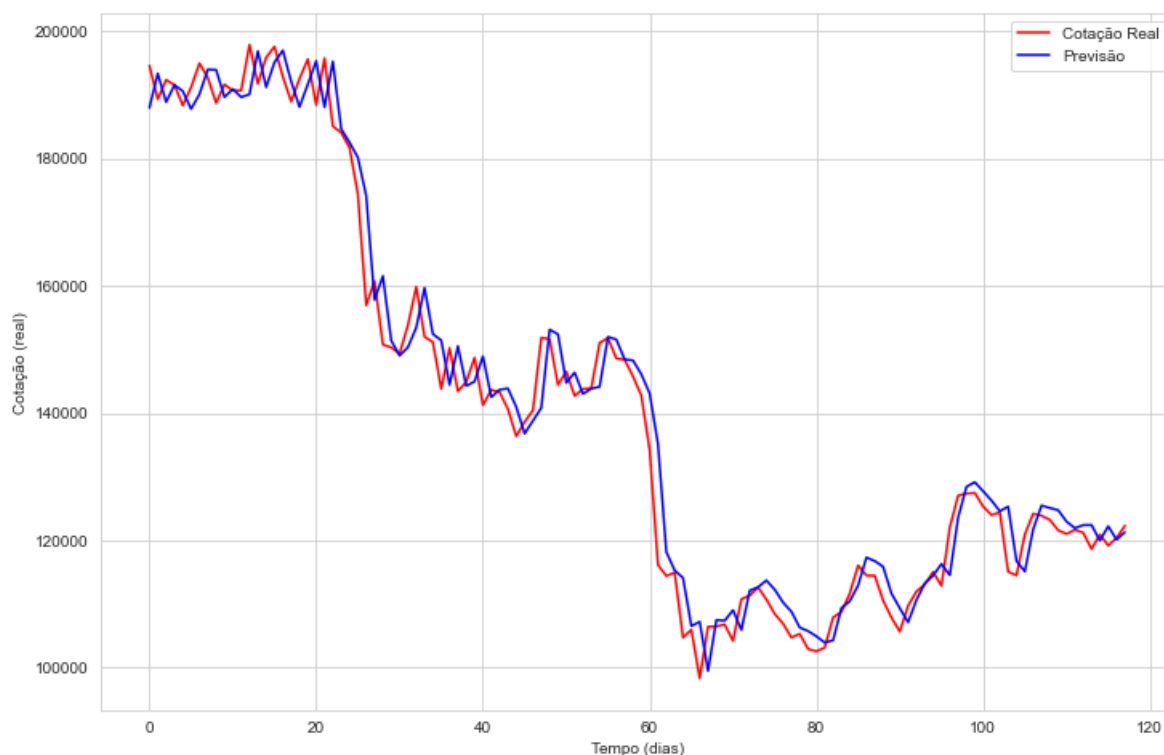
Portanto, para fins de comparação, os próximos modelos executados (tanto LSTM, quanto RNN) terão as configurações de tamanho de lote 16, otimizador *adam* e tamanho de janela 3. Tais modelos serão executados 5 vezes com sementes diferentes (1, 42, 74, 101, 139), e cada resultado será a média das 5 execuções, além de ser mostrado o desvio padrão dos dados apresentados.

5.4.3 Modelo com saída única para diferentes horizontes

Seguindo a configuração de parâmetros de menor taxa de erro do experimento anterior (otimizador *adam*, *batch* 16 e tamanho de janela de 3 dias), foram feitos testes para os horizontes de 1, 3, 5, 7, 9, 11, 13 e 15 dias usando o modelo LSTM e RNN tradicional de saída única. Neste caso, o modelo é treinado independentemente para cada um dos horizontes. Foram feitas 5 execuções e a média dos resultados obtidos são mostrados na [Tabela 3](#) e na [Tabela 4](#).

Além disso, na [Tabela 5](#) e [Tabela 6](#) são mostrados os desvios padrão das 5 execuções de cada modelo.

Figura 21 – Modelo RNN - Preço real vs previsão



Fonte: Desenvolvido pelo autor.

Tabela 2 – Resultados dos modelos RNN tradicional com saída única para horizonte 1

Tamanho de lote	Otimizador	Tamanho de janela	MAE	MAPE	RMSE	Tempo de execução (s)
32	adam	3	3579.968882	0.026046	4887.602220	12.497865
16	adam	3	3602.674523	0.026220	4882.207330	20.893112
64	adam	3	3819.448557	0.028271	5178.949199	8.182019
16	rmsprop	3	4175.126986	0.031178	5840.749986	27.228853
32	adam	7	4188.818790	0.031260	5751.454229	24.342763
32	rmsprop	3	5619.964645	0.042130	7704.092190	16.173843
64	adam	7	6328.238745	0.048179	7872.159167	15.046336
16	adam	7	6341.016552	0.049349	7746.287498	38.139307
16	adam	15	16801.577695	0.131152	20410.319393	72.229944
64	rmsprop	7	26572.792836	0.204066	32323.982411	17.110019
64	adam	15	39378.888573	0.309592	46024.273336	28.694596
32	rmsprop	7	43064.940016	0.349659	62234.882622	27.163373
32	rmsprop	15	56705.577993	0.451749	63449.491696	48.947801
16	rmsprop	7	66797.149894	0.514208	70736.445573	44.975019
64	rmsprop	3	70091.105403	0.572088	81352.379227	9.952912
64	rmsprop	15	80805.950113	0.623766	84762.139137	30.682348
32	adam	15	83456.676861	0.700338	182862.577330	46.081194
16	rmsprop	15	116677.103747	0.851514	128755.037425	78.724096

Fonte: Desenvolvido pelo autor.

Percebe-se que os resultados não foram bons para horizontes acima de 1 dia, uma vez que o erro cresce bastante. O modelo LSTM desempenhou melhor para 1, 3 e 7 dias, enquanto a RNN tradicional saiu melhor para os outros horizontes. Além disso, outro fator é que o desvio

Tabela 3 – Médias dos modelos LSTM com saída única para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.

	MAE	MAPE	RMSE	Tempo de execução (s)
Horizonte				
1	3595.994333	0.026113	4942.148159	81.224710
3	7223.040221	0.055238	9706.214169	80.632041
5	11810.943215	0.089913	15741.007498	79.667506
7	16127.200516	0.127261	20524.702111	78.257572
9	31211.005533	0.254416	36376.506607	78.973869
11	43907.903360	0.362413	51509.735355	78.523101
13	38197.169589	0.307858	42883.961320	79.525649
15	36566.779109	0.290953	40826.630126	79.603636

Fonte: Desenvolvido pelo autor.

Tabela 4 – Médias dos modelos RNN tradicional com saída única para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.

	MAE	MAPE	RMSE	Tempo de execução (s)
Horizonte				
1	3646.387765	0.026477	5006.728810	20.703774
3	9062.442807	0.069538	11384.628244	20.623218
5	10012.653235	0.077045	13558.960995	20.706722
7	16536.862249	0.129649	20928.334333	20.293078
9	17374.085533	0.136978	22351.525291	20.039049
11	25608.924819	0.201914	29942.452835	19.991254
13	26598.713208	0.208163	31857.430516	20.051718
15	31342.485111	0.248192	36008.097233	20.041397

Fonte: Desenvolvido pelo autor.

padrão das execuções variam bastante para horizontes acima de 1 dia, isso pode ser indicativo de que a rede não está gerando resultados confiáveis.

5.4.4 Modelo com múltiplas saídas

Para o treinamento desse modelo foi utilizada a mesma configuração obtida como melhor para o modelo de saída única (tamanho de *batch* 16, adam como otimizador e tamanho de janela 3) usada no modelo anterior, porém, trocando a última camada para rede de um para 8 neurônios, para prever múltiplas saídas simultaneamente, com os horizontes 1, 3, 5, 7, 9, 11, 13 e 15, respectivamente.

A Tabela 7 e Tabela 8 mostram as médias dos resultados dos modelos de saída múltipla para os diferentes horizontes. O tempo de execução médio (s) para a LSTM foi de 80.01 segundos e para a RNN tradicional foi de 20.64 segundos. Os desvios padrão das execuções podem ser

Tabela 5 – Desvio padrão dos modelos LSTM com saída única para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.

	MAE	MAPE	RMSE
Horizonte			
1	53.302210	0.000394	122.574280
3	871.589450	0.007257	892.706196
5	1694.906792	0.010966	1697.920923
7	1361.216887	0.011380	1180.366690
9	10348.450050	0.088865	12834.431628
11	3517.970352	0.029714	4519.408829
13	5595.608852	0.046124	5583.531705
15	4523.639569	0.038351	4293.065420

Fonte: Desenvolvido pelo autor.

Tabela 6 – Desvio padrão dos modelos RNN tradicional com saída única para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.

	MAE	MAPE	RMSE
Horizonte			
1	51.360121	0.000387	82.701196
3	3900.321392	0.030445	3711.706542
5	1623.388542	0.012373	1724.599150
7	3743.885618	0.030313	3417.751735
9	4050.090350	0.030407	3662.584446
11	7977.606754	0.060677	7173.283781
13	8542.477932	0.067425	7209.750150
15	6153.178505	0.049706	4922.874748

Fonte: Desenvolvido pelo autor.

vistos na [Tabela 9](#) e na [Tabela 10](#).

Neste tipo de modelo os resultados para LSTM não foram bons, todos os horizontes possuem erros altos e os desvios padrão das execuções também são altos, indicando que os resultados não são confiáveis e a rede possa ou ter sofrido sobreajuste ou não ter aprendido. Na RNN tradicional, mesmo que o erro para horizonte de 1 dia tenha piorado em relação ao modelo de saída única, o erro para os outros horizontes e os desvios padrão foram bem melhores comparados à rede LSTM de múltiplas saídas, indicando mais confiança para a RNN tradicional neste caso.

Tabela 7 – Médias dos modelos LSTM com saída múltipla para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.

	MAE	MAPE	RMSE
Horizonte			
1	9692.955371	0.078240	12315.441430
3	15957.367375	0.131310	20282.591914
5	26565.309582	0.220408	33899.910270
7	36232.629021	0.304229	47261.151684
9	43182.638850	0.363547	56137.246583
11	43854.137094	0.367838	55279.033669
13	40047.043607	0.333768	48579.754010
15	43566.546974	0.361821	51682.360109

Fonte: Desenvolvido pelo autor.

Tabela 8 – Médias dos modelos RNN tradicional com saída múltipla para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.

	MAE	MAPE	RMSE
Horizonte			
1	5891.294306	0.042941	7749.235716
3	7458.231115	0.056625	10230.244669
5	10151.264408	0.077340	12934.878704
7	13252.955326	0.102759	17156.732349
9	13680.808564	0.106832	18621.322213
11	14004.480236	0.110033	19215.354161
13	15080.300278	0.119407	20521.323974
15	17472.976818	0.140050	23869.192481

Fonte: Desenvolvido pelo autor.

Tabela 9 – Desvio padrão dos modelos LSTM com saída múltipla para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.

	MAE	MAPE	RMSE
Horizonte			
1	3138.013505	0.026769	4304.152338
3	2423.662423	0.021192	3338.180378
5	3037.388903	0.026627	4137.645896
7	3913.854370	0.033908	5115.571957
9	4550.303329	0.039190	5908.588527
11	5448.871715	0.046499	6820.321374
13	5425.662035	0.045581	6474.760434
15	6242.572788	0.051526	6955.154519

Fonte: Desenvolvido pelo autor.

Tabela 10 – Desvio padrão dos modelos RNN tradicional com saída múltipla para horizontes 1, 3, 5, 7, 9, 11, 13 e 15.

	MAE	MAPE	RMSE
Horizonte			
1	723.719913	0.005086	1056.696727
3	1278.442060	0.009998	1704.518254
5	1511.886913	0.011917	1447.294587
7	2742.821147	0.022028	3299.607301
9	2139.500066	0.016491	2361.499863
11	484.818375	0.003676	742.797251
13	530.615223	0.004362	879.871623
15	483.800828	0.004580	431.584212

Fonte: Desenvolvido pelo autor.

CONCLUSÕES

De todos os modelos apresentados, o que obteve melhor resultado foi a rede LSTM de saída única para horizonte de 1 dia, com 2,58% de MAPE e 2,61% de MAPE médio para 5 execuções com sementes diferentes. A RNN tradicional para este mesmo caso também teve um bom resultado, com 2,60% de MAPE e 2,64% de MAPE médio para 5 execuções.

Em conclusão, a análise dos modelos de redes neurais apresentados revelou que a rede LSTM de saída única obteve melhores resultados para horizontes curtos, enquanto a RNN tradicional teve um desempenho superior para horizontes mais longos. Além disso, o modelo de RNN tradicional com múltiplas saídas demonstrou ser o mais adequado para a predição de longo prazo, superando até mesmo a rede LSTM de múltiplas saídas. No entanto, é importante destacar que cada modelo apresenta particularidades e é necessário realizar mais testes e ajustes individuais nos parâmetros e configurações de cada modelo para maximizar seu desempenho.

Com base nessas análises, a utilização de redes neurais em previsões de séries temporais apresenta-se como uma ferramenta promissora, porém exige cuidado e atenção na sua implementação e ajuste para garantir precisão e confiabilidade dos resultados.

REFERÊNCIAS

AGGARWAL, C. C. **An Introduction to Neural Networks**. Cham: Springer International Publishing, 2018. 1–52 p. ISBN 978-3-319-94463-0. Disponível em: <https://doi.org/10.1007/978-3-319-94463-0_1>. Citado na página 38.

ANTONPOULOS, A. **Mastering Bitcoin 2e**. Sebastopol, CA: O'Reilly Media, 2017. Citado na página 21.

AWOKE, T.; ROUT, M.; MOHANTY, L.; SATAPATHY, S. C. Bitcoin price prediction and analysis using deep learning models. In: **Communication Software and Networks**. Springer Singapore, 2020. p. 631–640. Disponível em: <https://doi.org/10.1007/978-981-15-5397-4_63>. Citado nas páginas 15 e 43.

BATISTA, G. E. d. A. P. A. **Pré-processamento de dados em aprendizado de máquina supervisionado**. Tese (Doutorado) — Universidade de São Paulo, São Carlos : Instituto de Ciências Matemáticas e de Computação, 2003. Citado nas páginas 47 e 48.

BIGTRENDS. **Japanese Candlestick Charting - History & Patterns**. 2015. Disponível em: <<https://www.bigtrends.com/education/japanese-candlestick-charting-history-patterns/>>. Citado na página 21.

BISWAS, S.; PAWAR, M.; BADOLE, S.; GALANDE, N.; RATHOD, S. Cryptocurrency price prediction using neural networks and deep learning. In: **2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS)**. IEEE, 2021. Disponível em: <<https://doi.org/10.1109/icaccs51430.2021.9441872>>. Citado nas páginas 15 e 44.

BOUZADA, M. A. C. Aprendendo decomposição clássica: Tutorial para um método de análise de séries temporais. **TAC – Tecnologias de Administração e Contabilidade**, ANPAD, v. 2, n. 1, p. 1–18, jun. 2012. Disponível em: <<https://doi.org/10.21714/2236-02632012v2n1tac14>>. Citado na página 24.

BOX, G. E. P.; JENKINS, G. M.; REINSEL, G. C. **Time series analysis**. 4. ed. Hoboken, NJ: Wiley-Blackwell, 2008. (Wiley Series in Probability and Statistics). Citado na página 29.

BOX, G. E. P.; JENKINS, G. M.; REINSEL, G. C.; LJUNG, G. M. 5th. ed. [S.l.]: Wiley. Citado na página 29.

BROCKWELL, P. J.; DAVIS, R. A. **Introduction to time series and forecasting**. 1996. ed. New York, NY: Springer, 2013. (Springer texts in statistics). Citado na página 22.

BROWNLEE, J. **Difference Between Classification and Regression in Machine Learning**. 2017. Disponível em: <<https://machinelearningmastery.com/classification-versus-regression-in-machine-learning/#:~:text=Classification%20vs%20Regression,-Classification%20predictive%20modeling&text=Classification%20is%20the%20task%20of,of%20predicting%20a%20continuous%20quantity.>> Citado nas páginas 24 e 25.

- CHATFIELD, C. **The Analysis of Time Series**. Chapman and Hall/CRC, 2003. Disponível em: <<https://doi.org/10.4324/9780203491683>>. Citado nas páginas 22 e 24.
- CLEVELAND, R. B.; CLEVELAND, W. S.; MCRAE, J. E.; TERPENNING, I. Stl: A seasonal-trend decomposition procedure based on loess (with discussion). **Journal of Official Statistics**, v. 6, p. 3–73, 1990. Citado na página 25.
- COINBASE. **O que é criptografia?** 2022. Disponível em: <<https://www.coinbase.com/pt/learn/crypto-basics/what-is-cryptography#:~:text=A%20rede%20Bitcoin%20emite%20a,que%20enviem%20bitcoin%20a%20você.>> Citado na página 20.
- DEBASTIANI, C. A. **Candlestick: um Método Para Ampliar Lucros na Bolsa de Valores**. second. [S.l.]: novatec, 2007. Citado nas páginas 21 e 22.
- DERBENTSEV, V.; DATSENKO, N.; BABENKO, V.; PUSHKO, O.; PURSKY, O. Forecasting cryptocurrency prices using ensembles-based machine learning approach. In: **2020 IEEE International Conference on Problems of Infocommunications. Science and Technology (PIC S&T)**. [S.l.]: IEEE, 2020. Citado na página 44.
- FLEISCHER, J. P.; LASZEWSKI, G. von; THERAN, C.; BAUTISTA, Y. J. P. Time series analysis of cryptocurrency prices using long short-term memory. **Algorithms**, MDPI AG, v. 15, n. 7, p. 230, jul. 2022. Disponível em: <<https://doi.org/10.3390/a15070230>>. Citado na página 45.
- GAO, Z.; HE, Y.; KURUOGLU, E. E. A hybrid model integrating LSTM and garch for bitcoin price prediction. In: **2021 IEEE 31st International Workshop on Machine Learning for Signal Processing (MLSP)**. [S.l.]: IEEE, 2021. Citado na página 44.
- HAYKIN, S. S. **Neural networks and learning machines**. Third. Upper Saddle River, NJ: Pearson Education, 2009. Citado nas páginas 33, 35, 36, 37 e 39.
- HILEMAN, G.; RAUCHS, M. 2017 global cryptocurrency benchmarking study. **SSRN Electronic Journal**, Elsevier BV, 2017. Disponível em: <<https://doi.org/10.2139/ssrn.2965436>>. Citado na página 15.
- HINTON G., S. N.; SWERSKY, K. **Neural networks for machine learning. lecture 6e**. 2012. University of Toronto. Citado na página 50.
- HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997. Citado na página 40.
- INDULKAR, Y. Time series analysis of cryptocurrencies using deep learning & fbprophet. In: **2021 International Conference on Emerging Smart Computing and Informatics (ESCI)**. IEEE, 2021. Disponível em: <<https://doi.org/10.1109/esci50559.2021.9397004>>. Citado nas páginas 15 e 44.
- IQBAL, M.; IQBAL, M.; JASKANI, F.; IQBAL, K.; HASSAN, A. Time-series prediction of cryptocurrency market using machine learning techniques. **EAI Endorsed Transactions on Creative Technologies**, European Alliance for Innovation n.o., v. 8, n. 28, p. 170286, ago. 2021. Disponível em: <<https://doi.org/10.4108/eai.7-7-2021.170286>>. Citado na página 43.

JAKUBOWICZ, E.; ABDELFAHATTAH, E. The rise and fall of bitcoin: Predicting market direction using machine learning models. In: **2021 IEEE 12th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)**. IEEE, 2021. Disponível em: <<https://doi.org/10.1109/iemcon53756.2021.9623119>>. Citado na página 45.

KAUSHIK, M.; GIRI, A. K. **Forecasting Foreign Exchange Rate: A Multivariate Comparative Analysis between Traditional Econometric, Contemporary Machine Learning and Deep Learning Techniques**. arXiv, 2020. Disponível em: <<https://arxiv.org/abs/2002.10247>>. Citado nas páginas 33 e 34.

KEOGH, E.; LIN, J. Clustering of time-series subsequences is meaningless: implications for previous and future research. **Knowledge and Information Systems**, Springer Science and Business Media LLC, v. 8, n. 2, p. 154–177, ago. 2005. Disponível em: <<https://doi.org/10.1007/s10115-004-0172-7>>. Citado na página 27.

KIM, D. E.; GOFMAN, M. Comparison of shallow and deep neural networks for network intrusion detection. In: **2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)**. [S.l.]: IEEE, 2018. Citado na página 33.

KINGMA, J. B. D. P. **Adam: A Method for Stochastic Optimization**. 2014. ArXiv:1412.6980. Citado na página 50.

LI, Y.; DAI, W. Bitcoin price forecasting method based on CNN-LSTM hybrid neural network model. **J. Eng. (Stevenage)**, Institution of Engineering and Technology (IET), v. 2020, n. 13, p. 344–347, jul. 2020. Citado na página 45.

LIM, B.; ZOHREN, S. Time-series forecasting with deep learning: a survey. **Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences**, The Royal Society, v. 379, n. 2194, p. 13 pages, feb 2021. Disponível em: <<https://doi.org/10.1098/rsta.2020.0209>>. Citado na página 30.

LUXBURG, U. von; SCHÖLKOPF, B. Statistical learning theory: Models, concepts, and results. In: **Handbook of the History of Logic**. Elsevier, 2011. p. 651–706. Disponível em: <<https://doi.org/10.1016/b978-0-444-52936-7.50016-1>>. Citado na página 30.

MONTGOMERY, D. C.; JENNINGS, C. L.; KULAHCI, M. **Introduction to time series analysis and forecasting**. 2. ed. Nashville, TN: John Wiley & Sons, 2015. (Wiley Series in Probability and Statistics). Citado nas páginas 22, 23 e 29.

NAKAMOTO, S. **Bitcoin: A peer-to-peer electronic cash system**. 2009. Disponível em: <<http://www.bitcoin.org/bitcoin.pdf>>. Citado nas páginas 15, 19 e 20.

NG, A. **CS229 Lecture notes - Supervised learning**. 2003. <https://see.stanford.edu/materials/aimlcs229/cs229-notes1.pdf>. Citado nas páginas 30 e 31.

NIELSEN, A. **Practical time series analysis**. Sebastopol, CA: O'Reilly Media, 2019. Citado na página 33.

OLAH, C. **Understanding LSTM Networks**. 2015. Disponível em: <<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>>. Acesso em: 1 dez. 2020. Citado nas páginas 40 e 41.

- PAI, A. **CNN vs. RNN vs. ANN – Analyzing 3 Types of Neural Networks in Deep Learning**. 2020. Disponível em: <<https://www.analyticsvidhya.com/blog/2020/02/cnn-vs-rnn-vs-mlp-analyzing-3-types-of-neural-networks-in-deep-learning/>>. Citado na página 34.
- PARMEZAN, A. R. S. **Predição de séries temporais por similaridade**. Tese (Doutorado), 2016. Disponível em: <<https://doi.org/10.11606/d.55.2016.tde-21112016-150659>>. Citado nas páginas 29 e 30.
- PEREIRA, G. C.; CUSTODIO, R. Redes neurais para regressão uni- e multivariada. **Revista Chemkeys**, Universidade Estadual de Campinas, v. 3, p. e021003, set. 2021. Disponível em: <<https://doi.org/10.20396/chemkeys.v3i00.15880>>. Citado nas páginas 38 e 39.
- PRABHAKARAN, S. **ARIMA Model – Complete Guide to Time Series Forecasting in Python**. 2021. Disponível em: <<https://www.machinelearningplus.com/time-series/arima-model-time-series-forecasting-python/>>. Citado na página 30.
- SAKTHIVEL, U.; SUSILA, N. **Applied learning algorithms for intelligent IoT**. London, England: CRC Press, 2021. Citado nas páginas 31 e 32.
- SIAMI-NAMINI, S.; NAMIN, A. S. Forecasting economics and financial time series: ARIMA vs. LSTM. **CoRR**, abs/1803.06386, 2018. Disponível em: <<http://arxiv.org/abs/1803.06386>>. Citado na página 30.
- SORJAMAA, A.; HAO, J.; REYHANI, N.; JI, Y.; LENDASSE, A. Methodology for long-term prediction of time series. **Neurocomputing**, Elsevier BV, v. 70, n. 16-18, p. 2861–2869, out. 2007. Disponível em: <<https://doi.org/10.1016/j.neucom.2006.06.015>>. Citado na página 26.
- SOUZA, R. C. Metodologias para a análise e previsão de séries temporais univariadas e multivariadas. **Brazilian Review of Econometrics**, Fundacao Getulio Vargas, v. 1, n. 2, p. 78, nov. 1981. Disponível em: <<https://doi.org/10.12660/bre.v1n21981.3167>>. Citado na página 26.
- SOVBETOV, Y. Factors influencing cryptocurrency prices: Evidence from bitcoin, ethereum, dash, bitcoin, and monero. **Journal of Economics and Financial Analysis**, v. 2, n. 2, p. 1–27, 2018. Disponível em: <<https://EconPapers.repec.org/RePEc:trp:01jefa:jefa0016>>. Citado na página 15.
- WU, C.-H.; LU, C.-C.; MA, Y.-F.; LU, R.-S. A new forecasting framework for bitcoin price with LSTM. In: **2018 IEEE International Conference on Data Mining Workshops (ICDMW)**. IEEE, 2018. Disponível em: <<https://doi.org/10.1109/icdmw.2018.00032>>. Citado nas páginas 15 e 43.